Object Oriented Analysis and Design Using the UML Version 4.2

Module 09A: Class Design

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

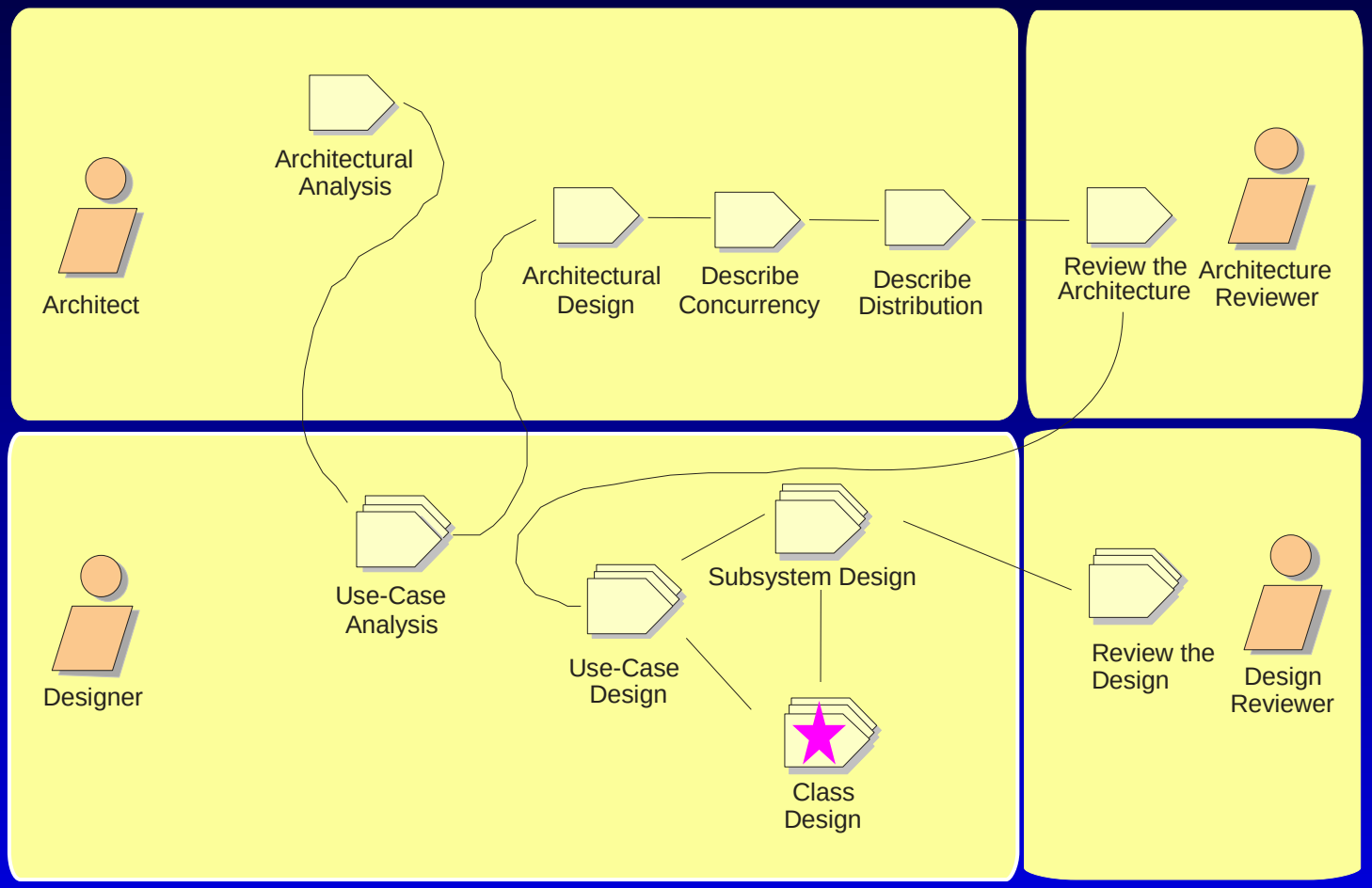
Objectives: Class Design

- Understand the purpose of Class Design and where in the lifecycle it is performed
- Identify additional classes and relationships needed to support implementation of the chosen architectural mechanisms
- Identify and analyze state transitions in objects of state-controlled classes
- Refine relationships, operations, and attributes

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

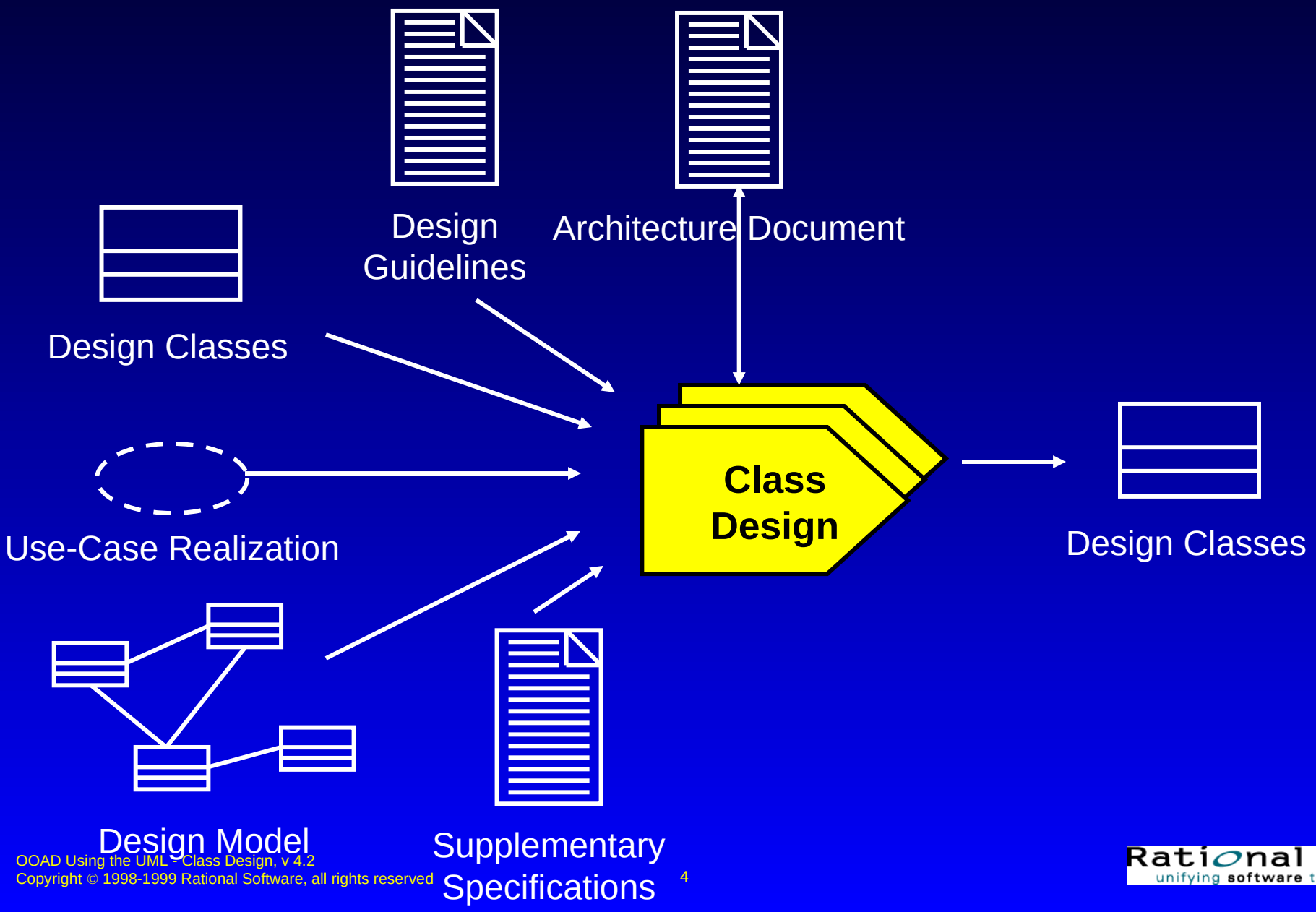
Class Design in Context



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Overview



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Considerations

- Class stereotype
  - Boundary
  - Entity
  - Control
- Applicable design patterns
- Architectural mechanisms
  - Persistence
  - Distribution
  - etc.

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

How Many Classes Are Needed?

- Many, simple classes means that each class
  - Encapsulates less of the overall system intelligence
  - Is more reusable
  - Is easier to implement
- A few, complex classes means that each class
  - Encapsulates a large portion of the overall system intelligence
  - Is less likely to be reusable
  - Is more difficult to implement

A class should have a single well focused purpose.

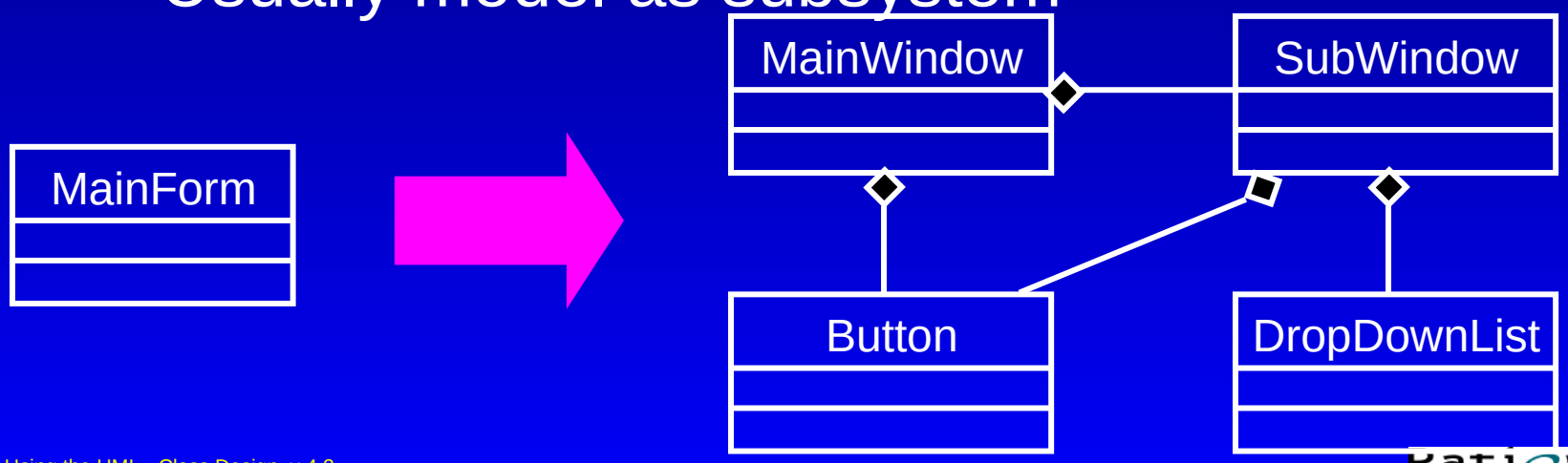
A class should do one thing and do it well!

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Designing Boundary Classes

- User interface (UI) boundary classes
  - What user interface development tools will be used?
  - How much of the interface can be created by the development tool?
- External system interface boundary classes
  - Usually model as subsystem



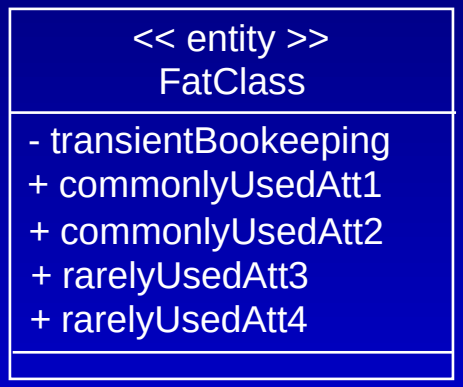
Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

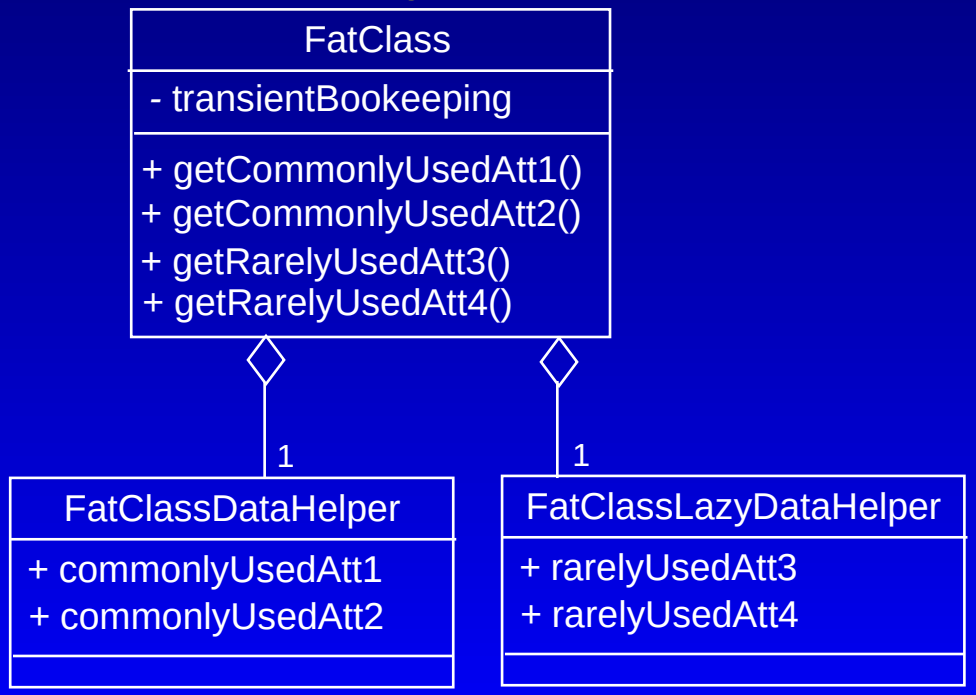
Designing Entity Classes

- Entity objects are often passive and persistent
- Performance requirements may force some re-factoring
- See Identify Persistent Classes step

Analysis



Design



Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Designing Control Classes

- What Happens to Control Classes?
  - Are they really needed?
  - Should they be split?
- How do you decide?
  - Complexity
  - Change probability
  - Distribution and performance
  - Transaction management

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

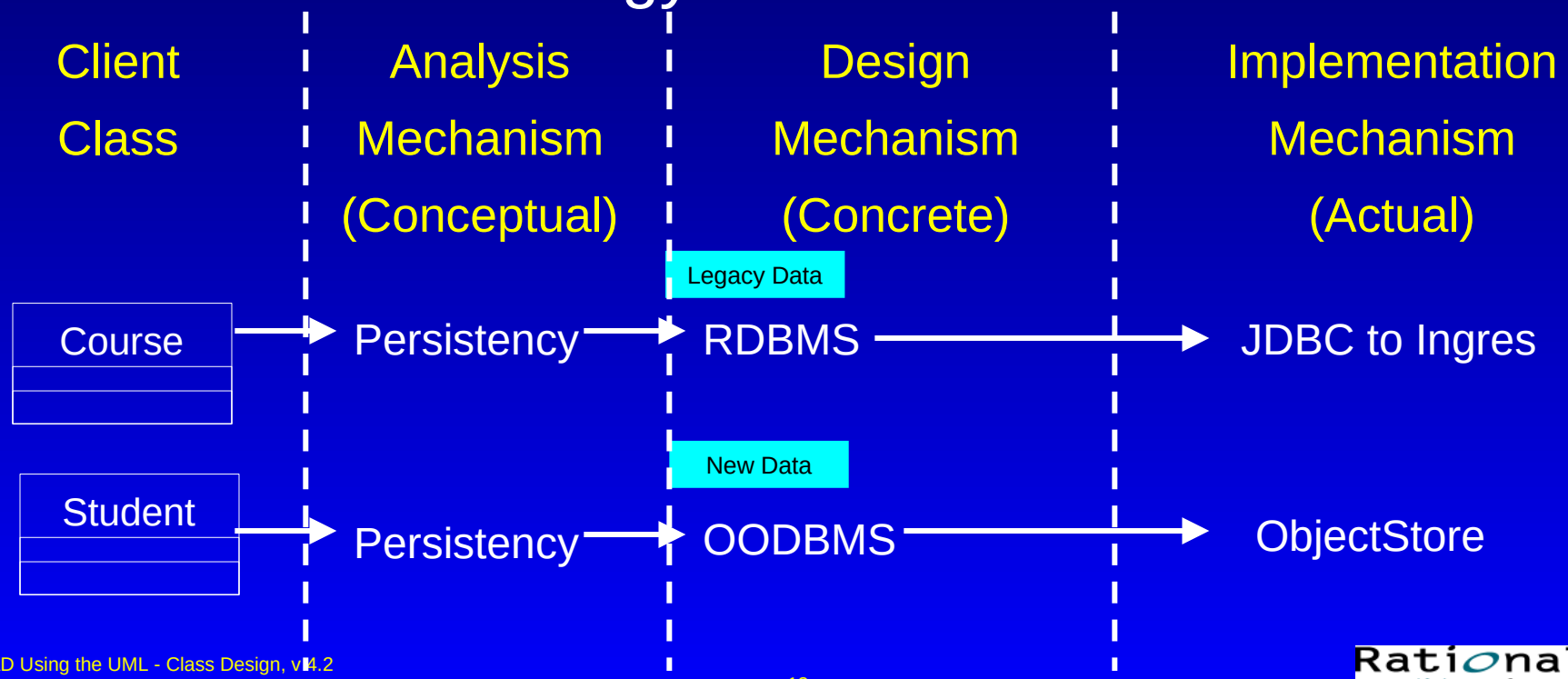
- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Identify Persistent Classes

- Any instance of the class requires its state to be preserved
- Persistent classes are mapped to the persistence mechanism
- Persistence strategy must be coordinated

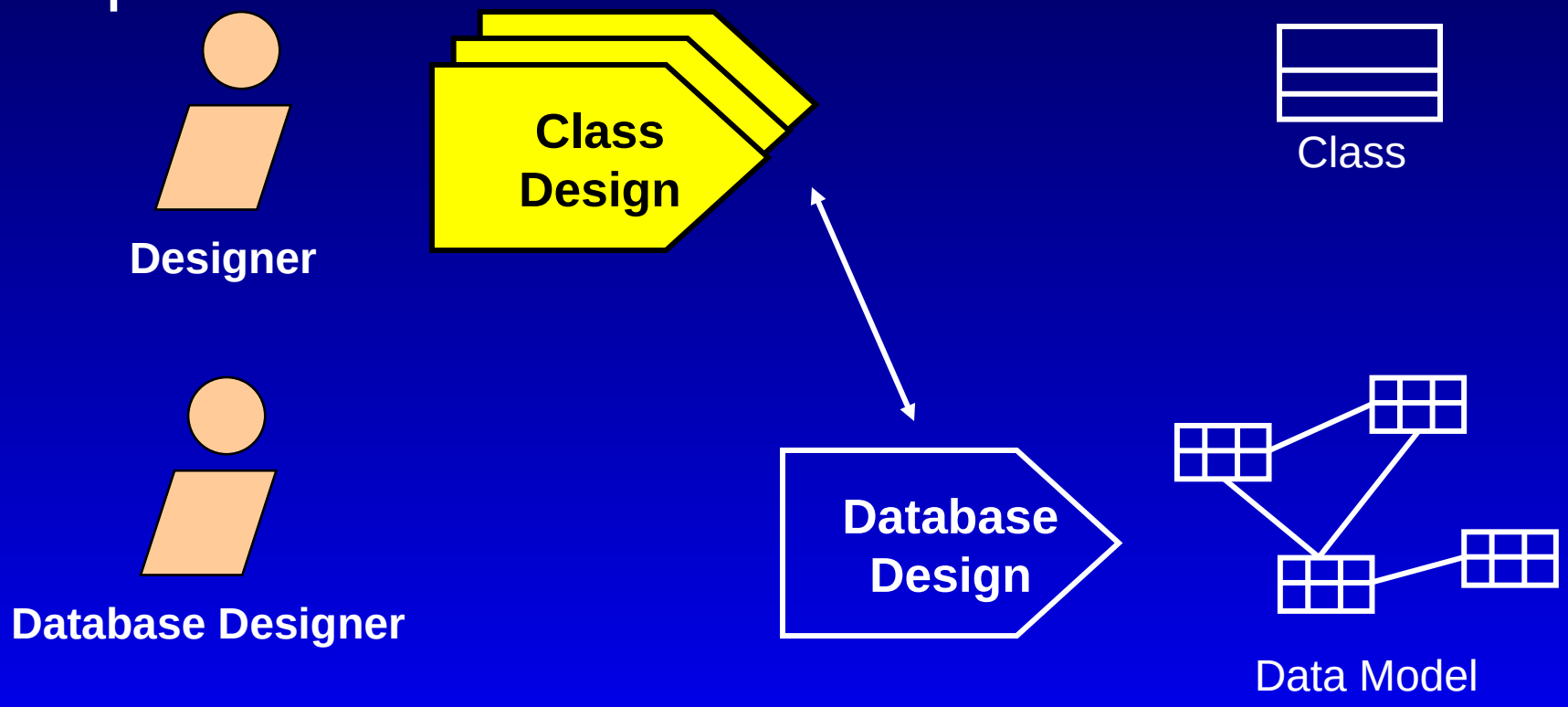


Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Database Design Preview

- Persistence strategy must be coordinated
- At this point, just indicate the classes are persistent



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

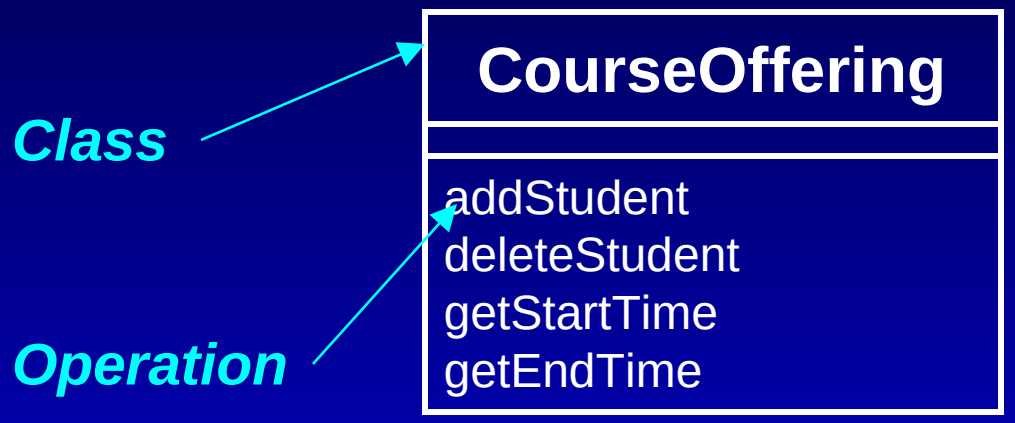
Define Operations

- Purpose
  - Map responsibilities defined in analysis to the operations that implement them
- Things to consider :
  - Operation name, signature, and description
  - Operation visibility
  - Operation scope
    - Class operation vs. instance operation

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Review: What is an Operation?

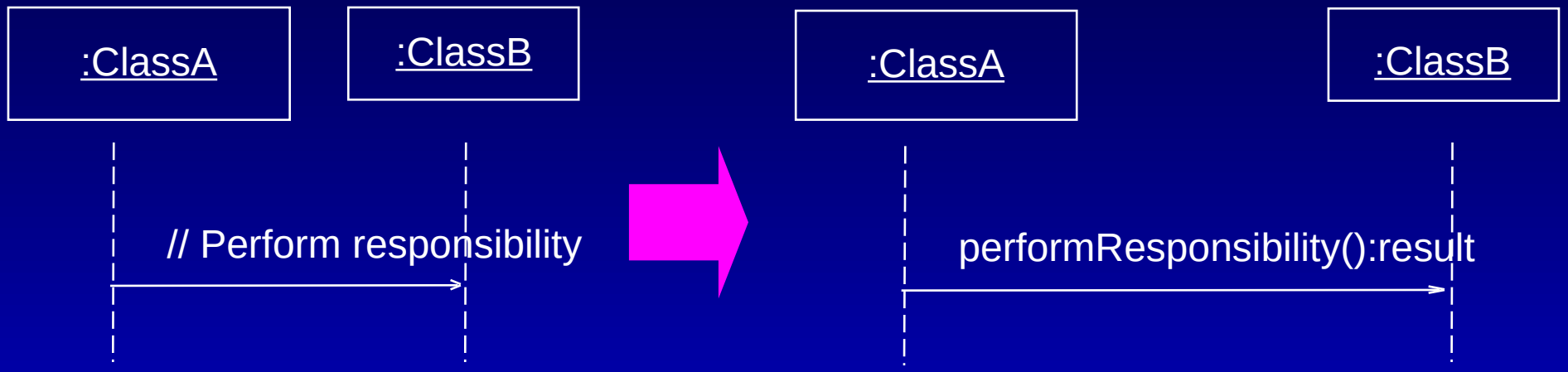


Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Operations: Where Do You Find Them?

- Messages displayed in interaction diagrams



- Other implementation dependent functionality
  - Manager functions
  - Need for class copies
  - Need to test for equality

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Name and Describe the Operations

- Appropriate operation names
  - Indicate the outcome
  - Use client perspective
  - Consistent across classes
- Define operation signatures
  - operationName(parameter : class,..) : returnType
- Provide short description, including meaning of all parameters

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

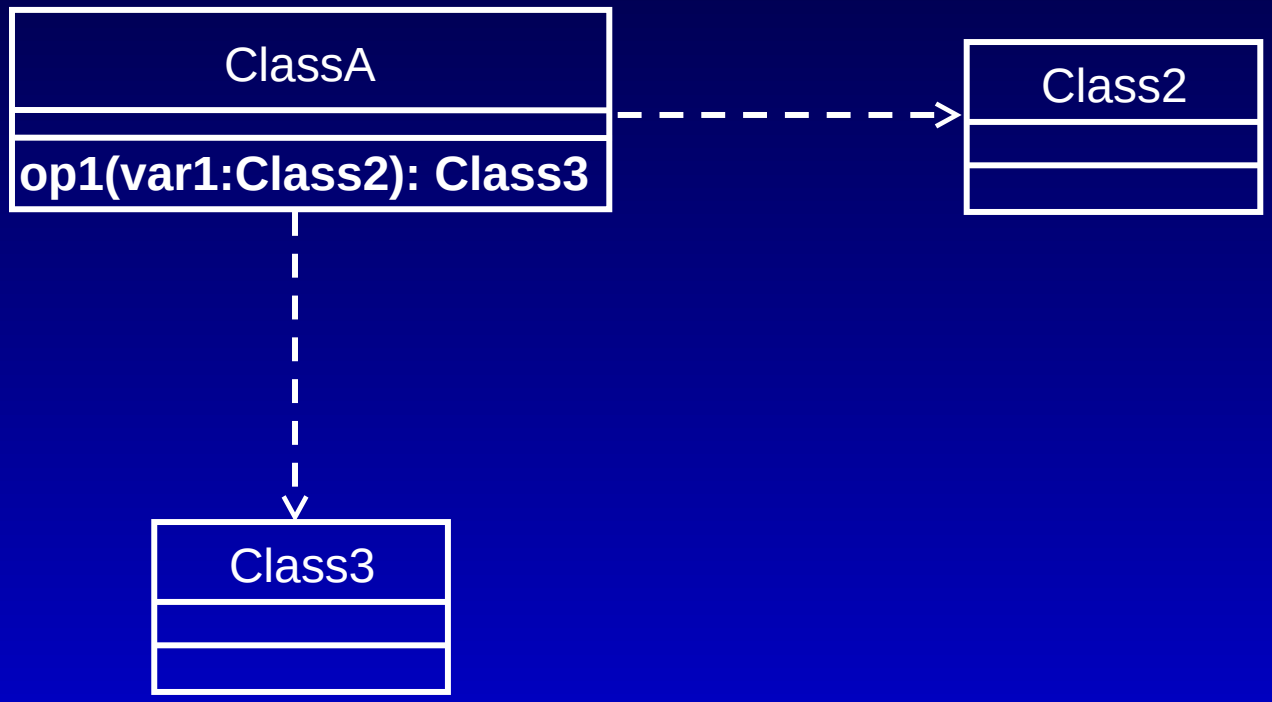
Guidelines: Designing Operation Signatures

- When designing operation signatures be sure to include:
  - Parameters passed by-value or by-reference?
  - Parameters changed by operation?
  - Parameters optional?
  - Default parameter values?
  - Valid parameter ranges?
- The fewer the parameters, the better
- Pass objects instead of “data bits”

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Discovering Additional Classes and Relationships



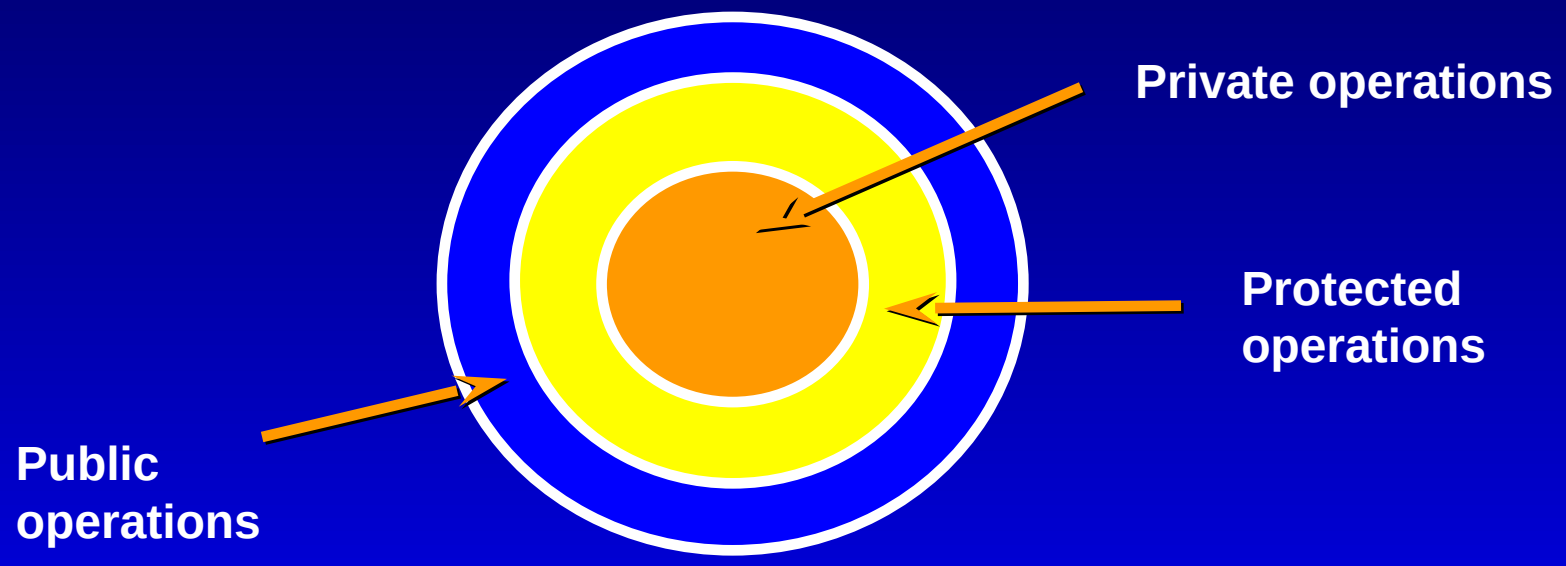
Additional classes and relationships may be added to support signature

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Operation Visibility

- Visibility is used to enforce encapsulation
- May be public, protected, or private



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

How Is Visibility Noted?

- The following symbols are used to specify export control:
  - +Public access
  - #Protected access
  - - Private access

Class
- privateAttribute # protectedAttribute
+publicOp() # protectedOp() - privateOp()

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Scope

- Determines number of instances of the attribute/operation
  - Instance: one instance for each class instance
  - Classifier: one instance for all class instances
- Classifier scope is denoted by underlining the attribute/operation name

Class
- <u>classifierScopeAttribute</u> - instanceScopeAttribute
<u>classifierScopeOperation</u>() instanceScopeOperation()

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Example: Scope

<<entity>>
Student

- name
- address
- studentID
- <u>nextAvailID : int</u>

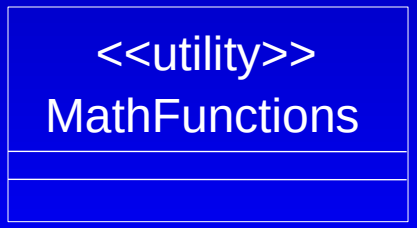
+ addSchedule(theSchedule : Schedule, forSemester : Semester)
+ getSchedule(forSemester : Semester) : Schedule
+ hasPrerequisites(forCourseOffering : CourseOffering) : boolean
# passed(theCourseOffering : CourseOffering) : boolean
+ <u>getNextAvailID() : int</u>

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Utility Classes

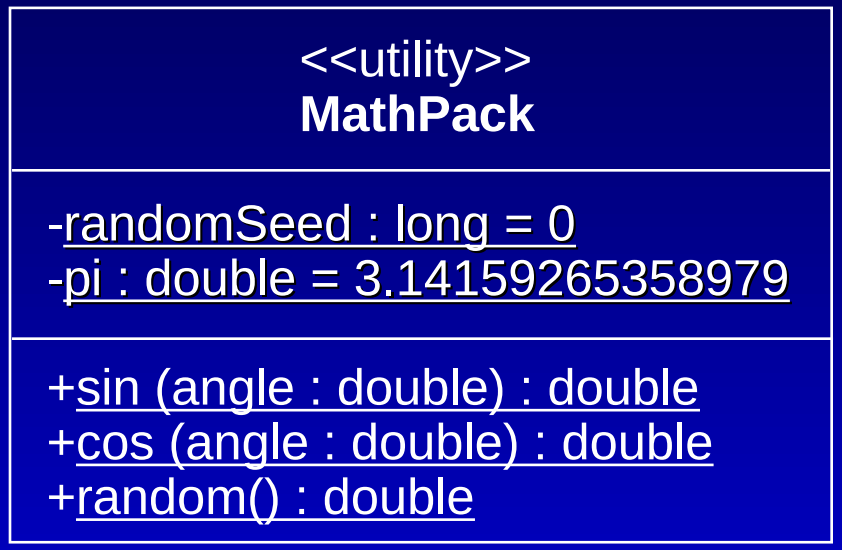
- What is a Utility Class?
  - Utility is a class stereotype
  - Used for a class that contains a collection of free subprograms
- Why use it?
  - To provide services that may be (re)useful in a variety of contexts
  - To wrap non object-oriented libraries or applications



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

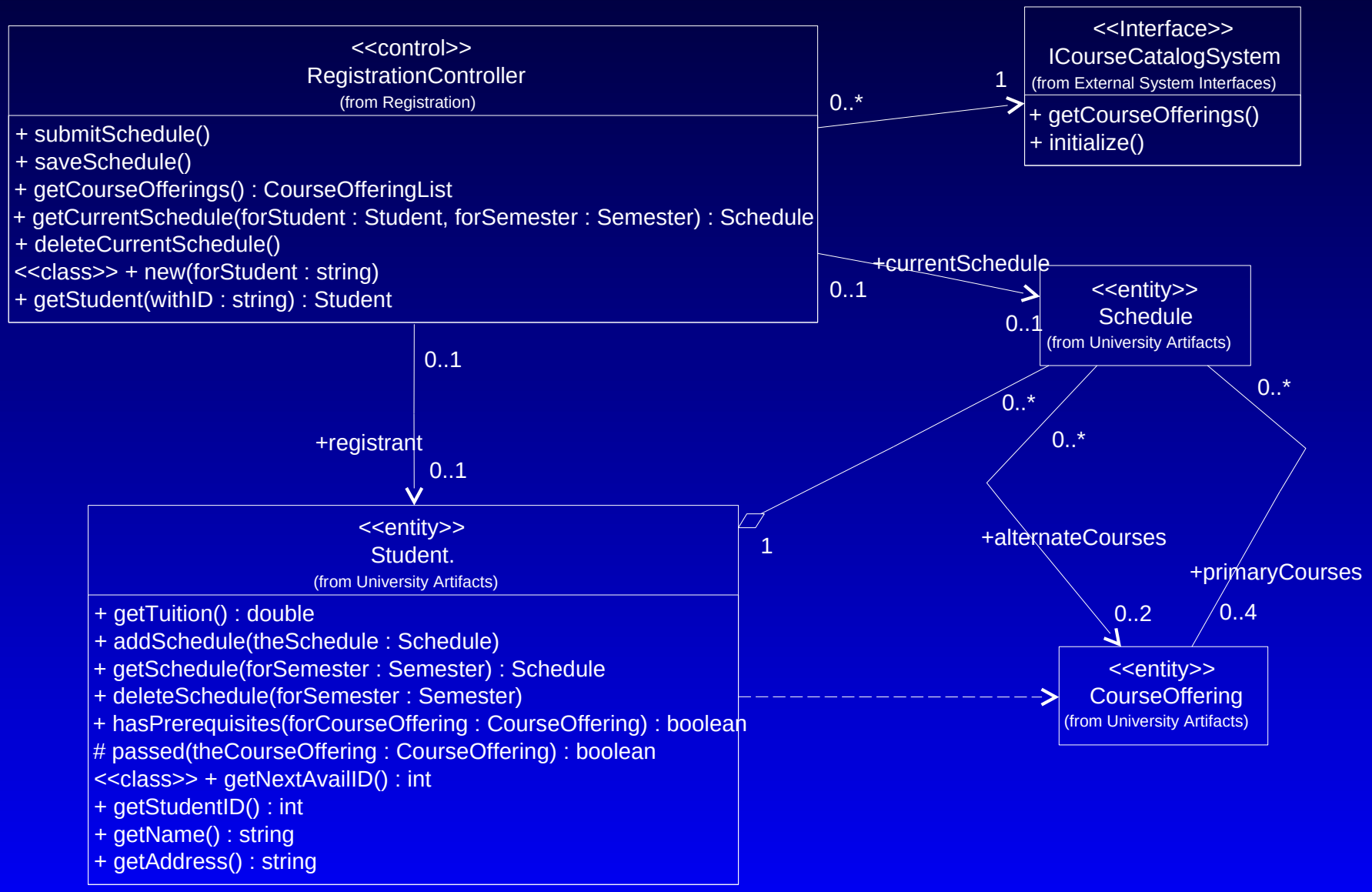
Example: Utility Classes



Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Example: Define Operations



Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Exercise: Define Operations

- Given the following:
  - The architectural layers, their packages, and their dependencies
  - Design classes for a particular use case

(continued)

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Exercise: Define Operations (cont.)

- Identify the following for the design classes:
  - Operations and complete operation signatures
  - Operation scope and visibility
  - Any additional relationships and/or classes to support the defined operations and operation signatures

(continued)

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Exercise: Define Operations (cont.)

- Produce the following diagrams:
  - VOPC class diagram, containing all operations, operation signatures, and relationships

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

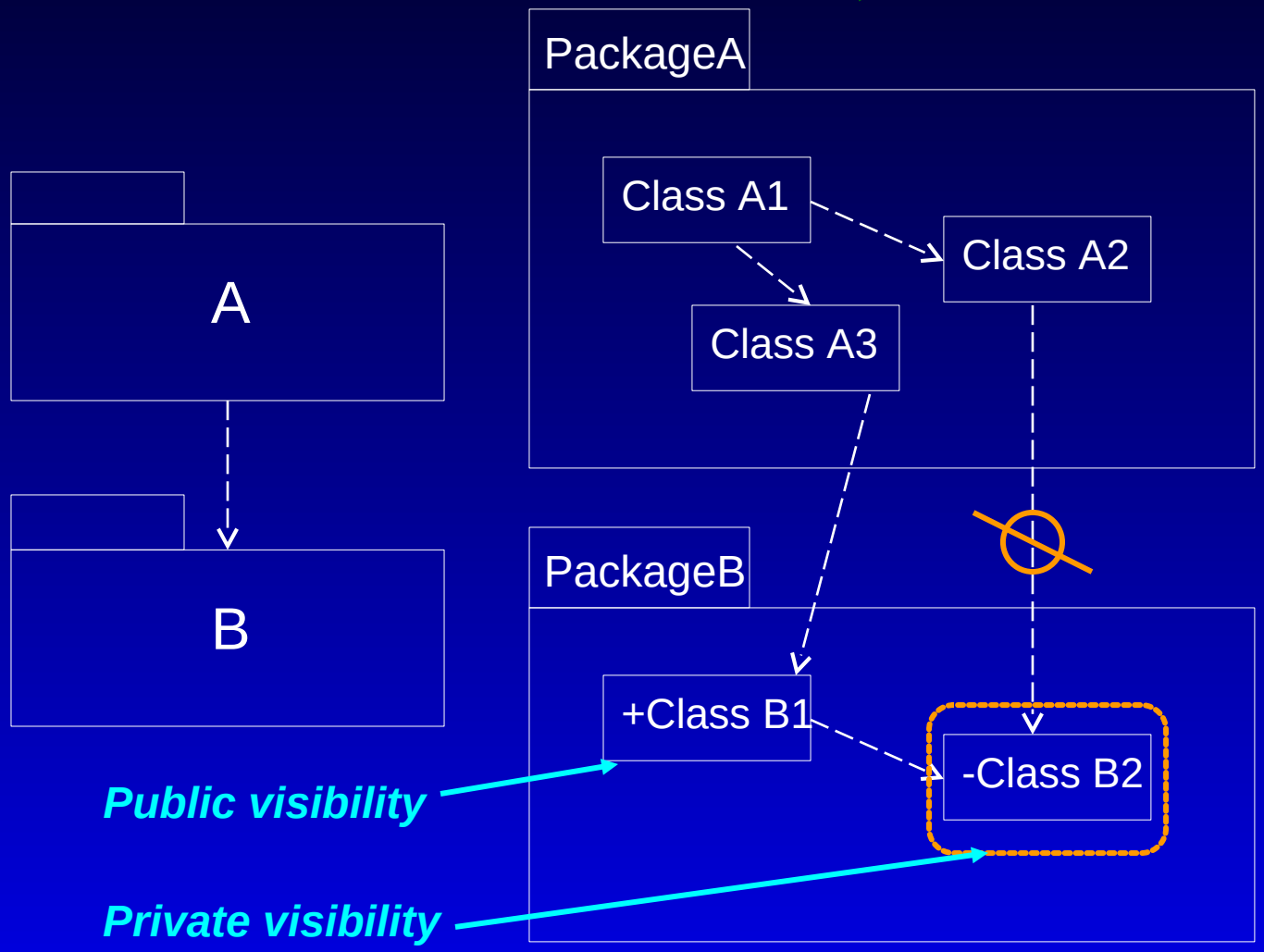
Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Review: Package Element Visibility



Only public classes can be referenced outside of the owning package

OO Principle: Encapsulation

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Define Methods

- What is a method?
  - Describes operation implementation
- Purpose
  - Define special aspects of operation implementation
- Things to consider :
  - Special algorithms
  - Other objects and operations to be used
  - How attributes and parameters are to be implemented and used
  - How relationships are to be implemented and used

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Define States

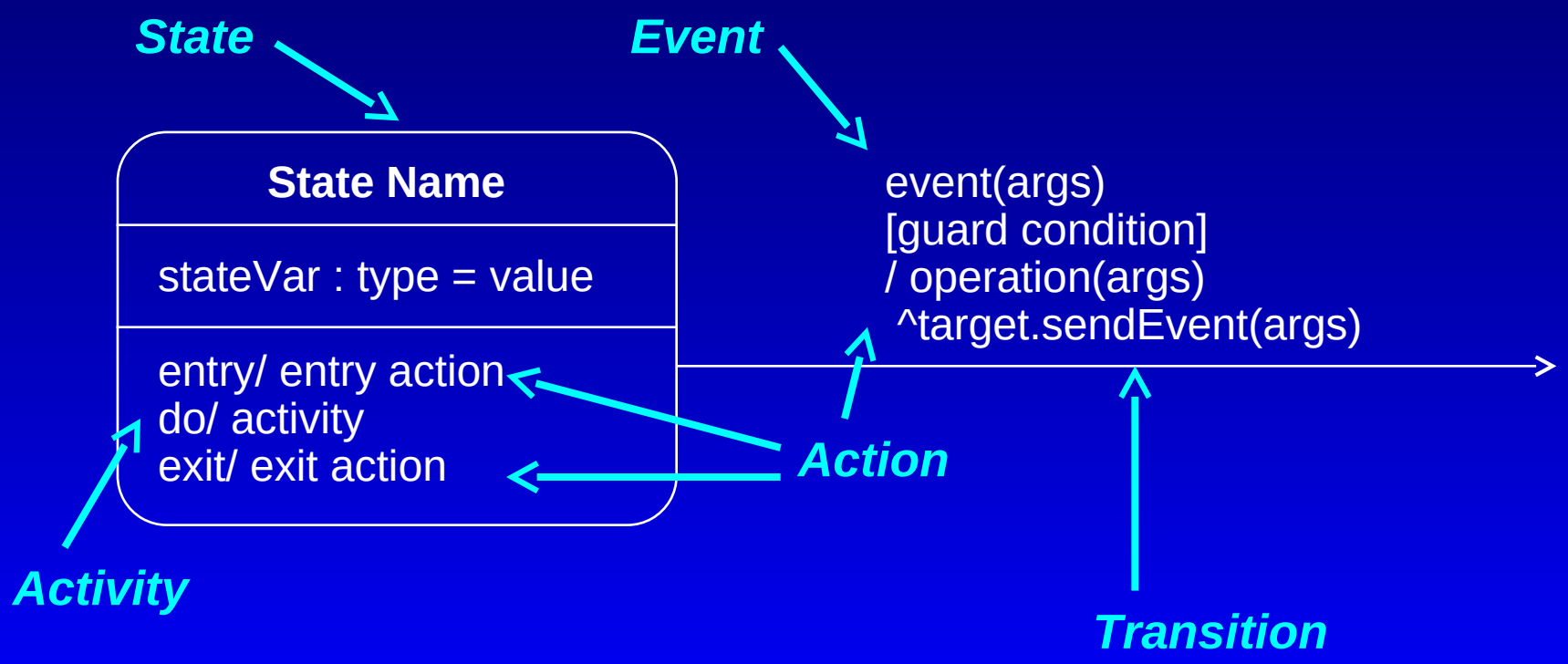
- Purpose
  - Design how an object's state affects its behavior
  - Develop statecharts to model this behavior
- Things to consider :
  - Which objects have significant state?
  - How to determine an object's possible states?
  - How do statecharts map to the rest of the model?

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

What is a Statechart?

- A directed graph of states (nodes) connected by transitions(directed arcs)
- Describes the life history of a reactive object

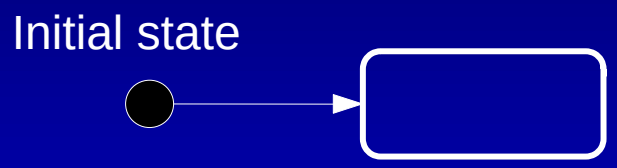


Copyright © 1998-1999 Rational Software, all rights reserved

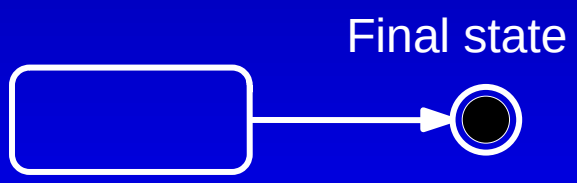
Rational unifying software teams

Special States

- Initial state
  - The state entered when an object is created
  - Mandatory
  - Can only have one initial state



- Final state
  - Indicates the object's end of life
  - Optional
  - May have more than one



Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Process For Deriving Statecharts

- Identify and define the states
- Identify the events
- Identify the transitions (i.e., the event responses)
- Add activities and actions

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Identify and Define the States

- Significant, dynamic attributes

The maximum number of students per course offering is 10

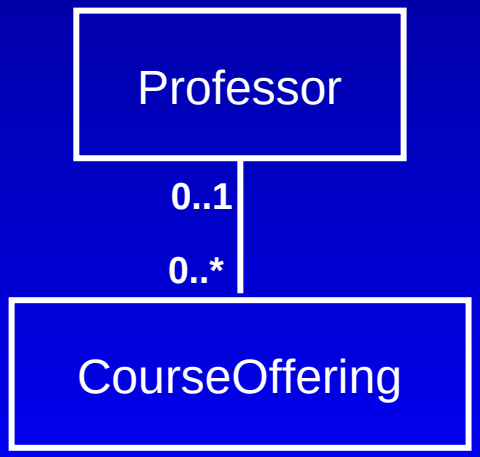
numStudents < 10

Open

numStudents > = 10

Closed

- Existence and non-existence of certain links



Link to Professor Exists

Assigned

Link to Professor Doesn't Exist

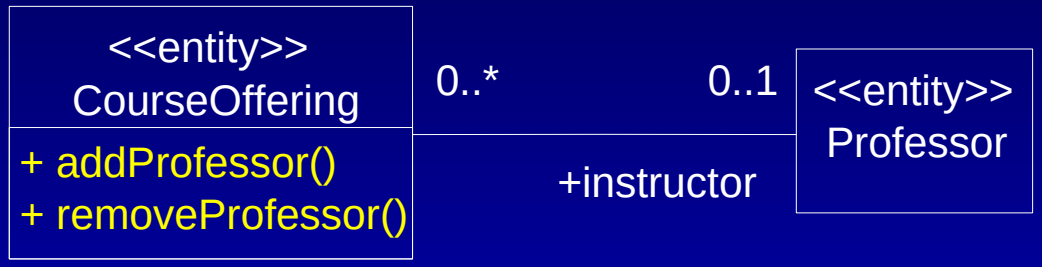
Unassigned

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Identify the Events

- Look at the class interface operations

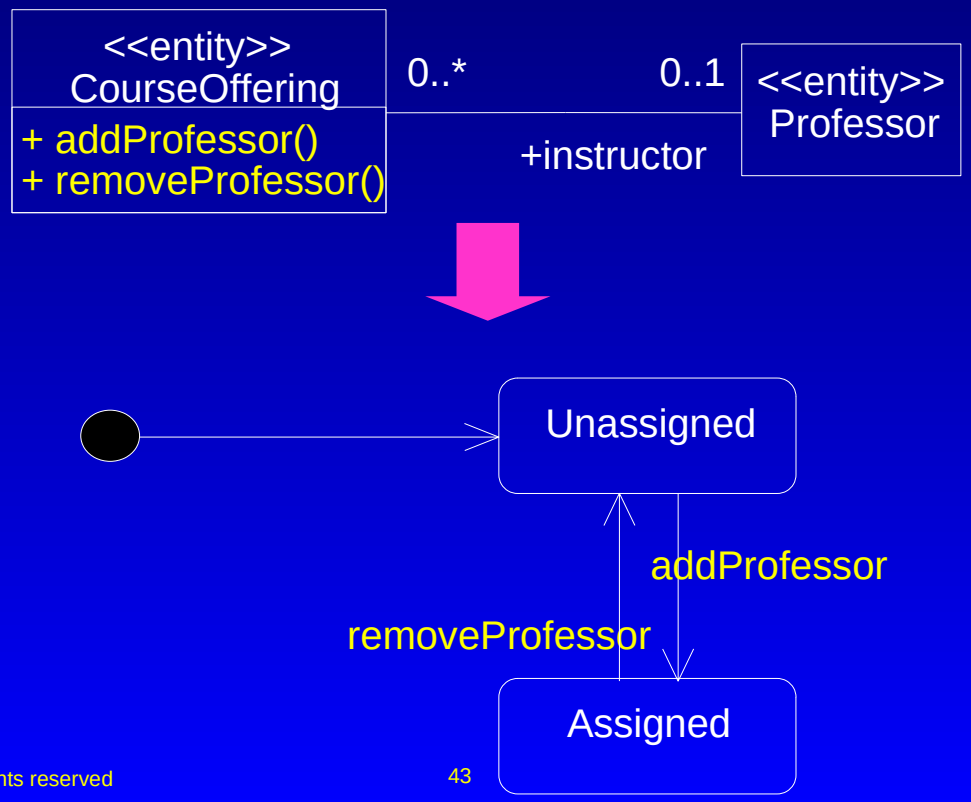


Events: addProfessor, removeProfessor

Copyright © 1998-1999 Rational Software, all rights reserved

Identify the Transitions

- For each state, determine what events cause transitions to what states, including guard conditions, when needed
- Transitions describe what happens in response to the receipt of an event

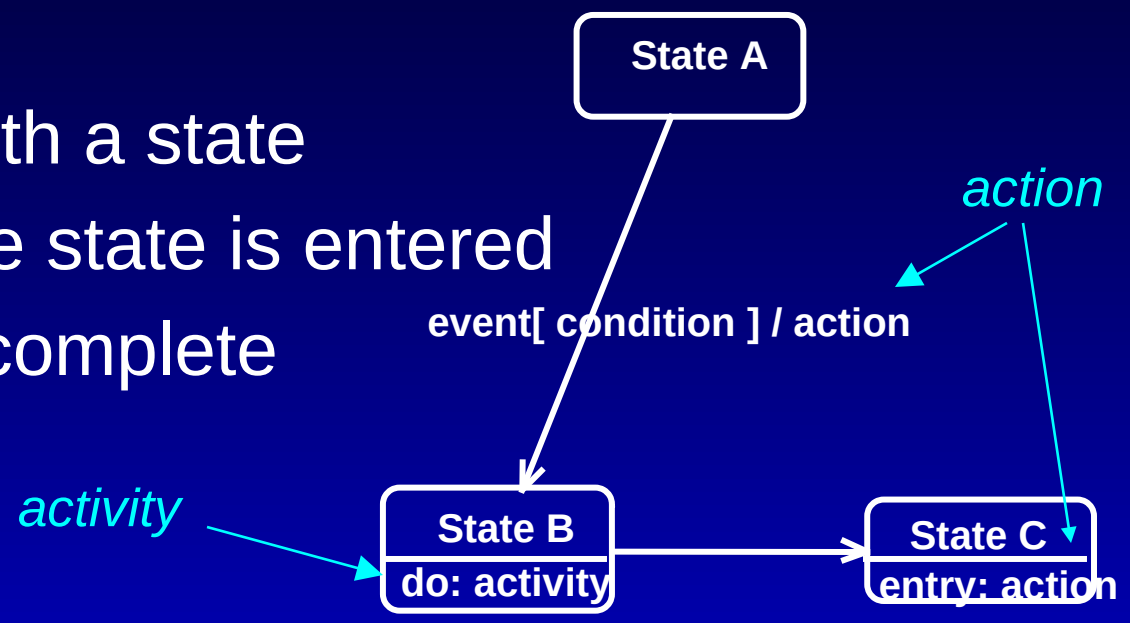


Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Add Activities and Actions

- Activities
  - Associated with a state
  - Start when the state is entered
  - Take time to complete
  - Interruptible
- Actions
  - Associated with a transition
  - Take an insignificant amount of time to complete
  - Non-interruptible

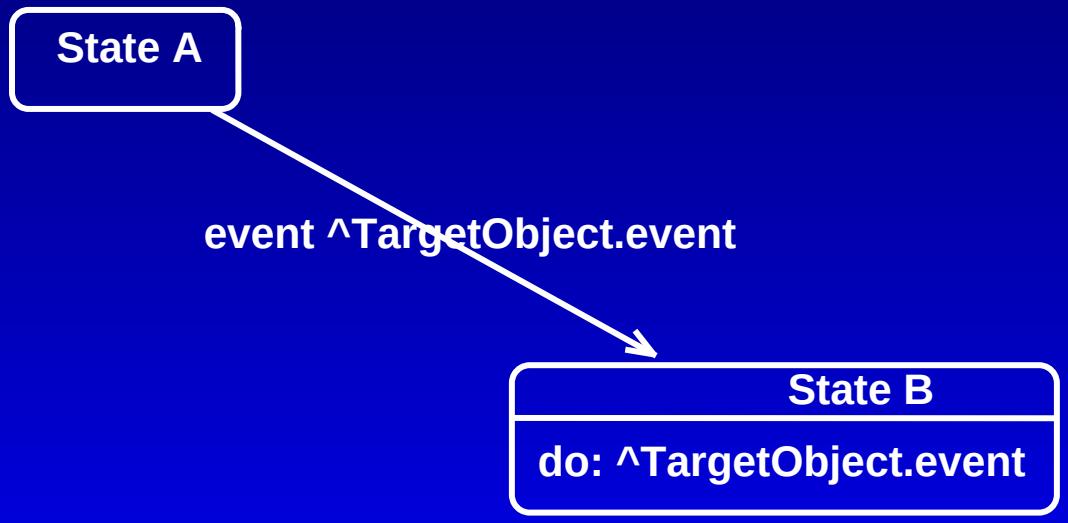


Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Send Events

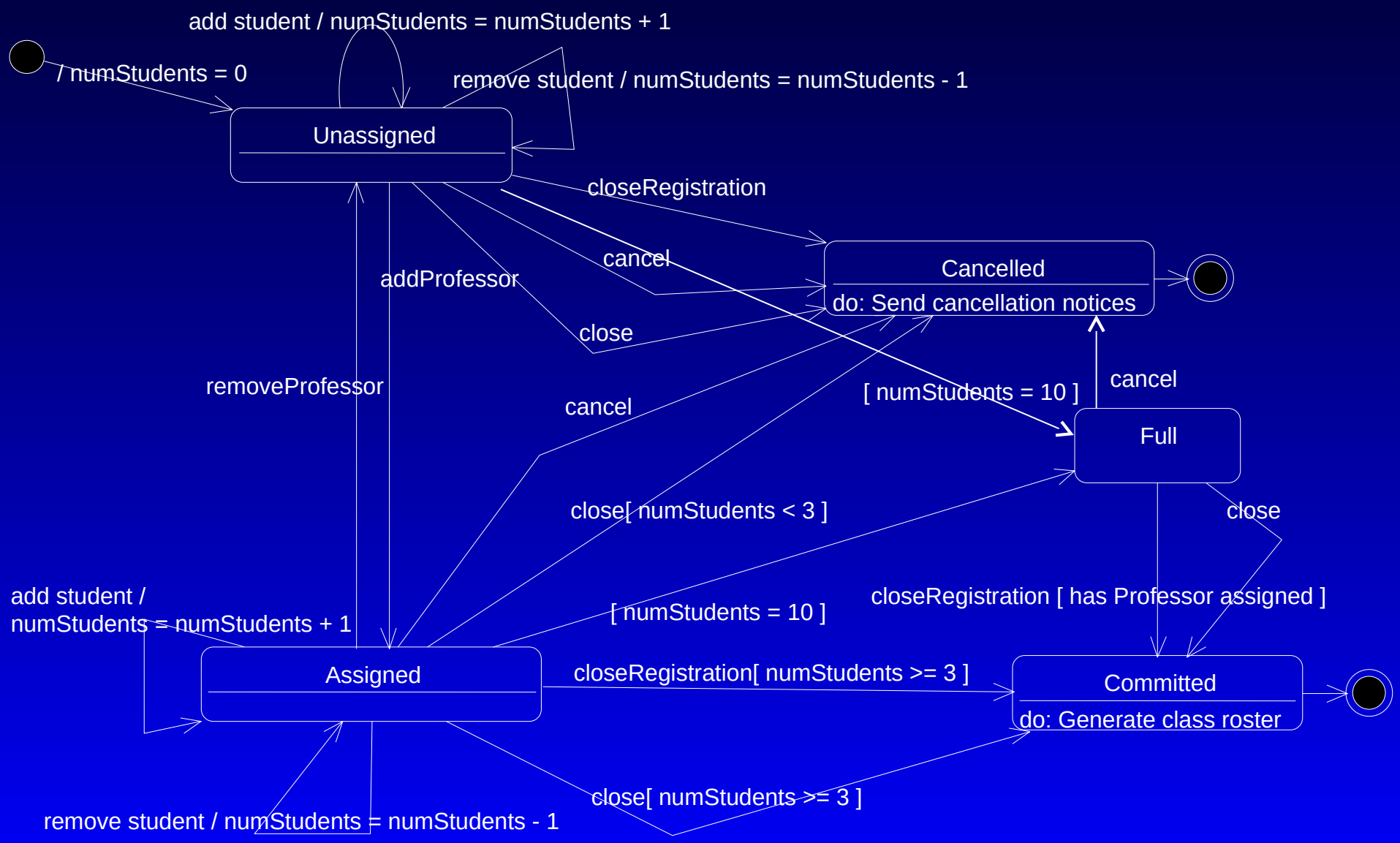
- One event may trigger the sending of another event
- An activity can also send an event to another object



Copyright © 1998-1999 Rational Software, all rights reserved

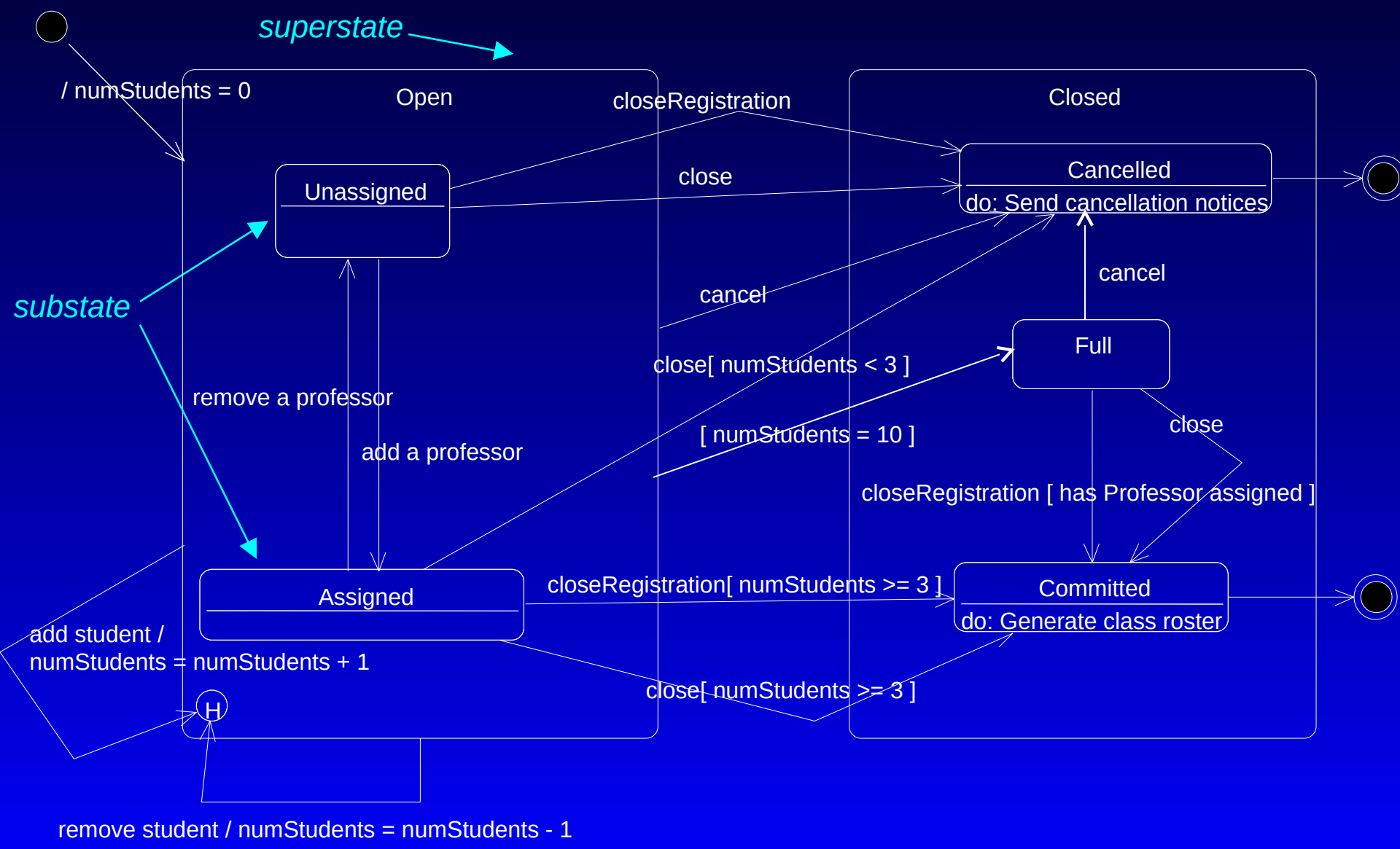
Rational unifying software teams

Example: Statechart



Copyright © 1998-1999 Rational Software, all rights reserved

Example: Statechart With Nested States and History



Copyright © 1998-1999 Rational Software, all rights reserved

Which Objects Have Significant State?

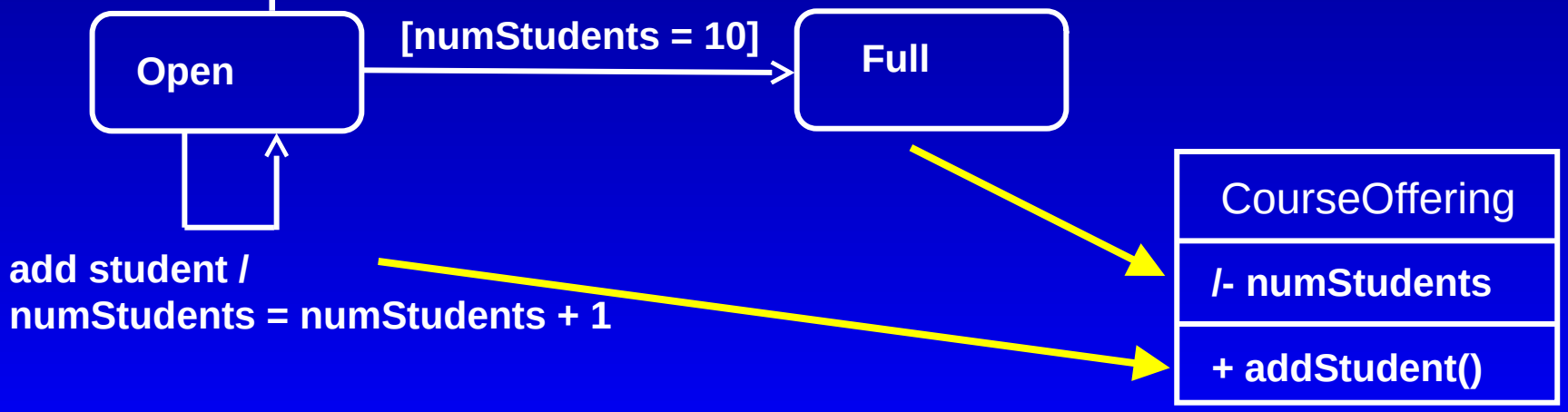
- Objects whose role is clarified by state transitions
- Complex use cases that are state-controlled
- It is not necessary to model all objects
  - Objects with straightforward mapping to implementation
  - Objects that are not state-controlled
  - Objects with only one computational state

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

How Do Statecharts Map to the Rest of the Model?

- Events may map to operations
- Methods should be updated with state-specific information
- States are often represented using attributes
  - This serves as input into the Define Attributes step



(Stay tuned for derived attributes)

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Exercise: Define States (optional)

- Given the following:
  - All design classes
- Identify the following:
  - Class(es) with significant state-controlled behavior
  - The important states and transitions for the identified class
- Produce the following diagrams:
  - Statechart for one of the classes that exhibits state-controlled behavior

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

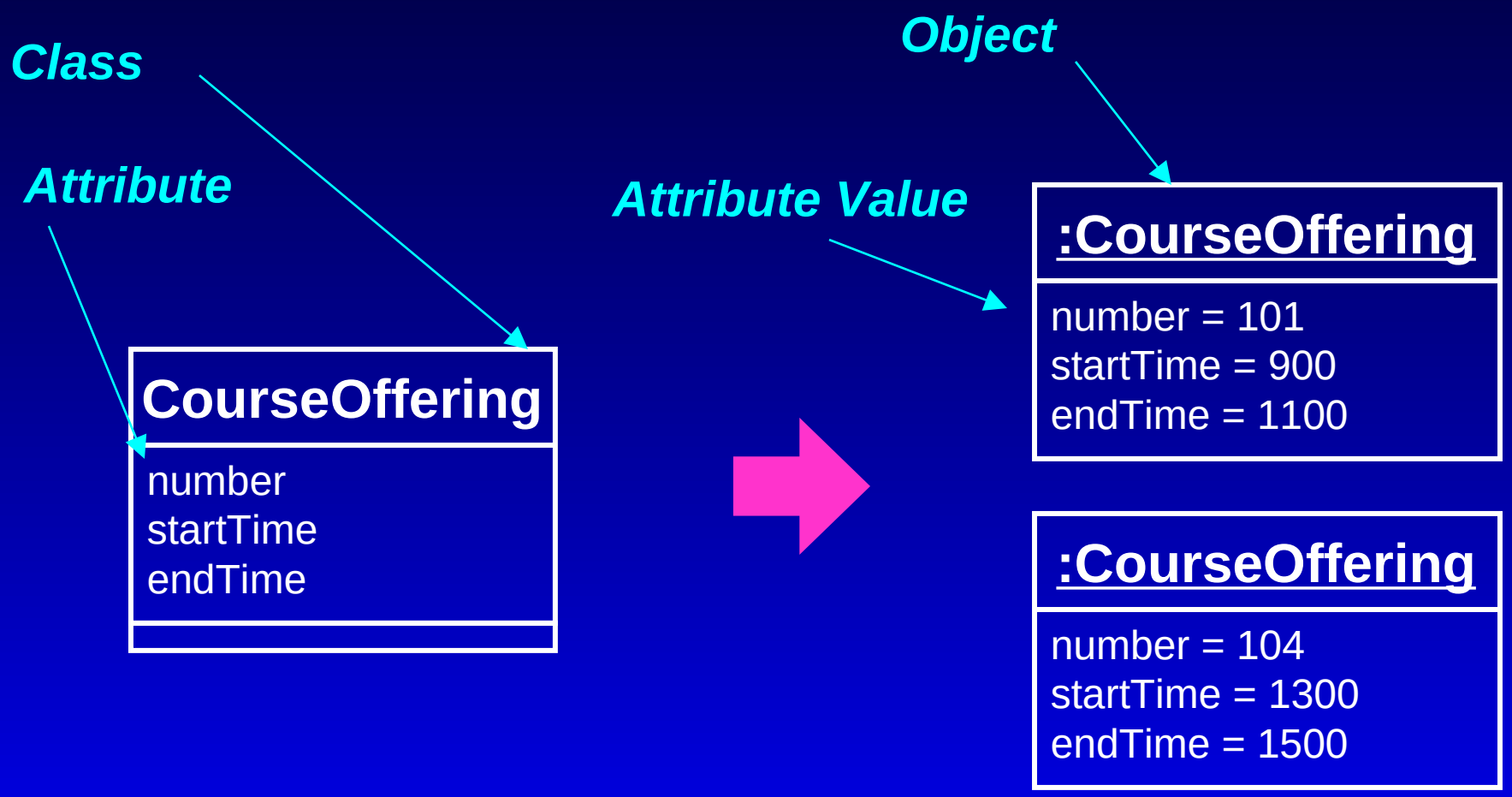
Define Attributes

- Purpose
  - Formalize definition of attributes
- Things to consider:
  - Persistency
  - Visibility
  - Name, type, and initial default value

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Review: What is an Attribute?



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Attributes: How Do You Find Them?

- Examine method descriptions
- Examine states
- Any information the class itself needs to maintain

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Attribute Representations

- Specify name, type, and optional default value
  - attributeName : Type = Default
- Follow naming conventions of implementation language and project
- Type should be an elementary data type in implementation language
  - Built-in data type, user-defined data type, or user-defined class
- Specify visibility
  - Public: ‘+’
  - Private: ‘-’
  - Protected: ‘#’

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

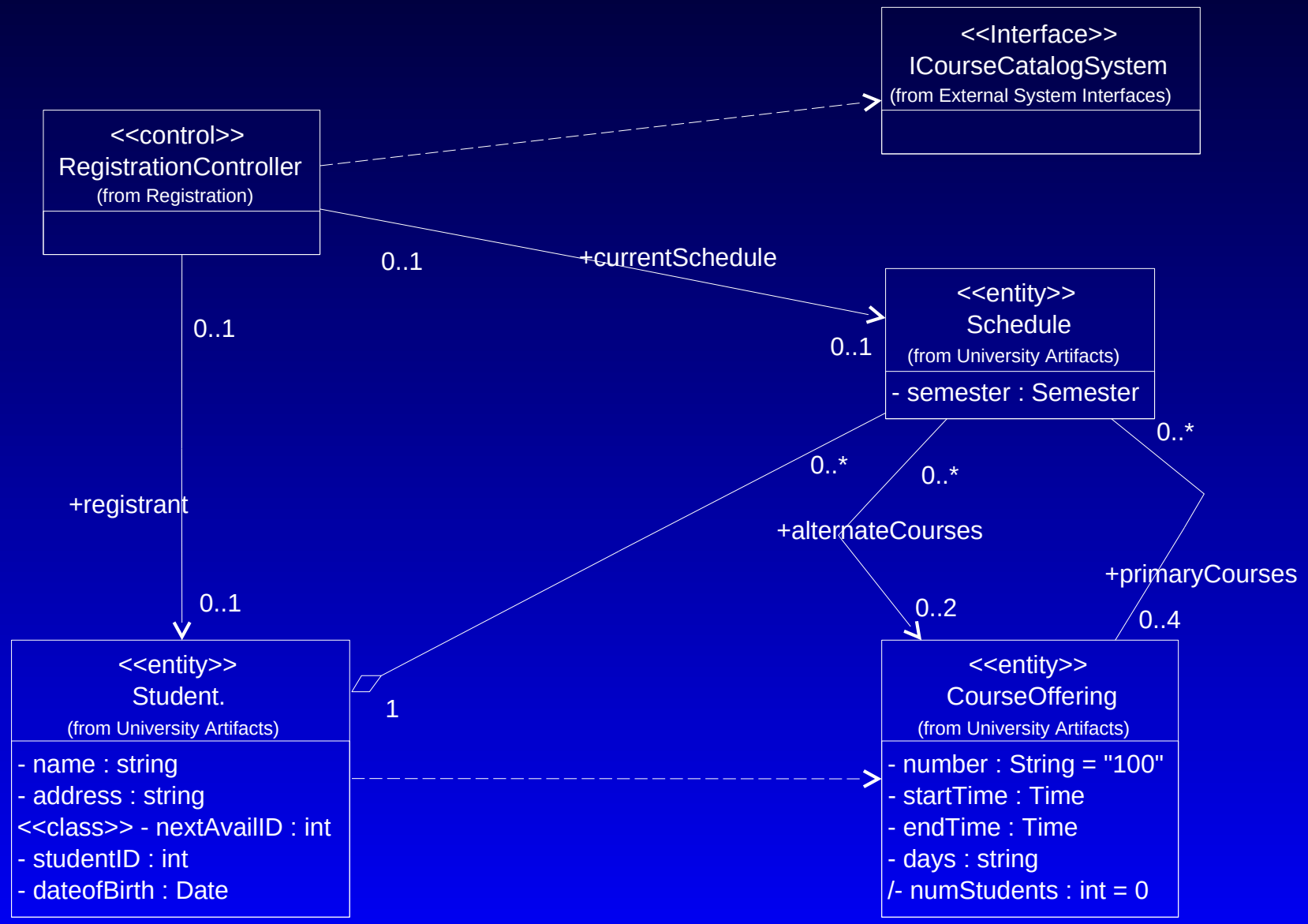
Derived Attributes

- What is a derived attribute?
  - An attribute whose value may be calculated based on the value of other attribute(s)
- When do you use them?
  - When there is not enough time to re-calculate the value every time it is needed
  - Trade-off runtime performance vs. memory required

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Example: Define Attributes



Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Exercise: Define Attributes (optional)

- Given the following:
  - The architectural layers, their packages, and their dependencies
  - Design classes for a particular use case

(continued)

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Exercise: Define Attributes (cont.)

- Identify the following for the design class:
  - Attributes and complete attribute signatures
  - Attribute scope and visibility
  - Any additional relationships and/or classes to support the defined attributes and attribute signatures

(continued)

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Exercise: Define Attributes (cont.)

- Produce the following diagrams:
  - VOPC class diagram, containing all attributes and relationships

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Define Dependency

- What Is a Dependency?
  - A relationship between two objects

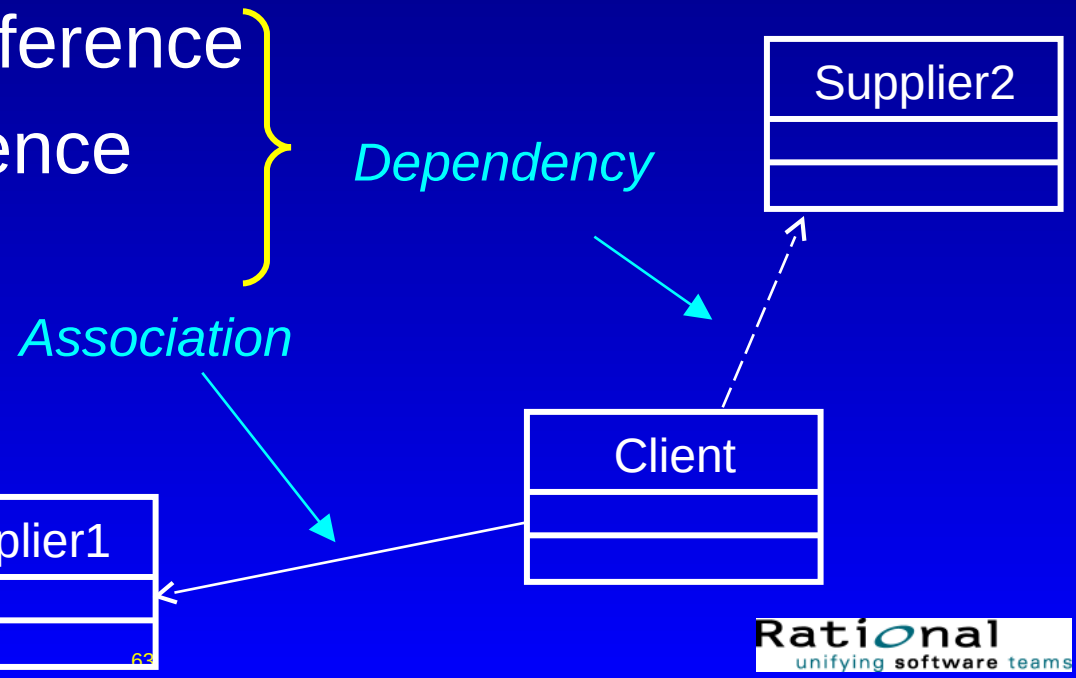
- Purpose
  - Determine where structural relationships are NOT required
- Things to look for :
  - What causes the supplier to be visible to the client

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Dependencies vs. Associations

- Associations are structural relationships
- Dependencies are non-structural relationships
- In order for objects to “talk” they must be visible
  - Local variable reference
  - Parameter reference
  - Global reference
  - Field reference

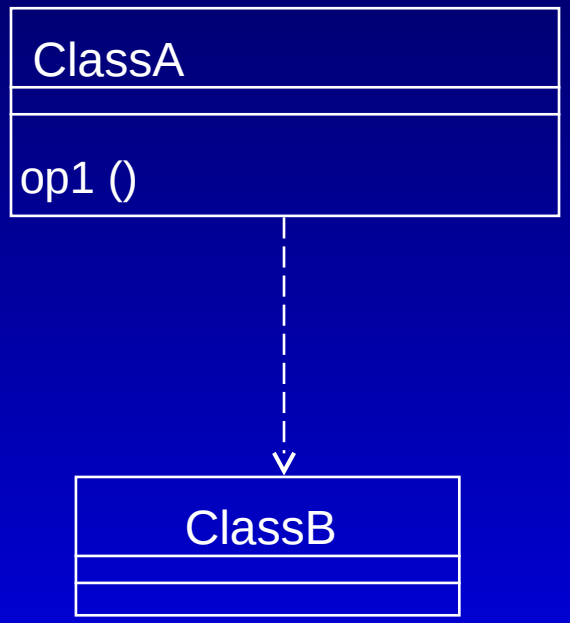


Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Local Variable Visibility

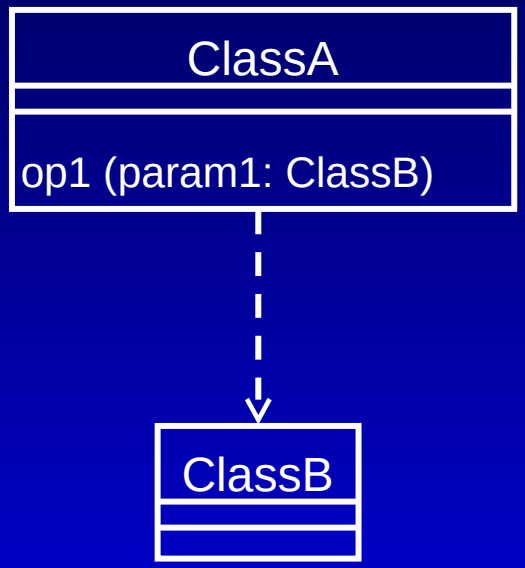
- The op1() operation contains a local variable of type ClassB



Copyright © 1998-1999 Rational Software, all rights reserved

Parameter Visibility

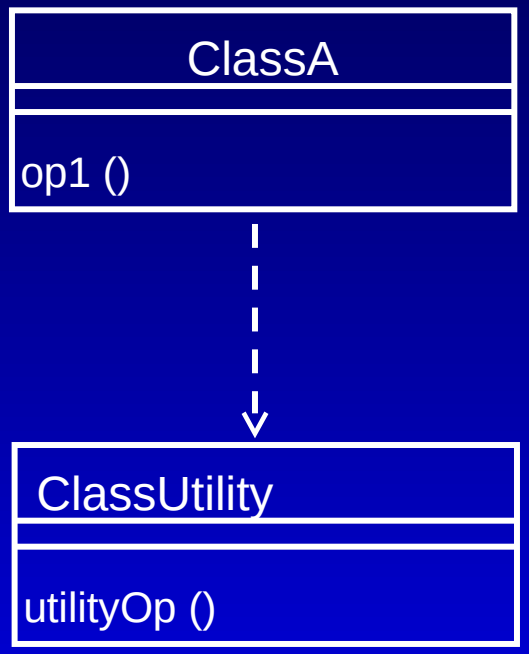
- The ClassB instance is passed to the ClassA instance



Copyright © 1998-1999 Rational Software, all rights reserved

Global Visibility

- The ClassUtility instance is visible because it is global



Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Identifying Dependencies: Considerations

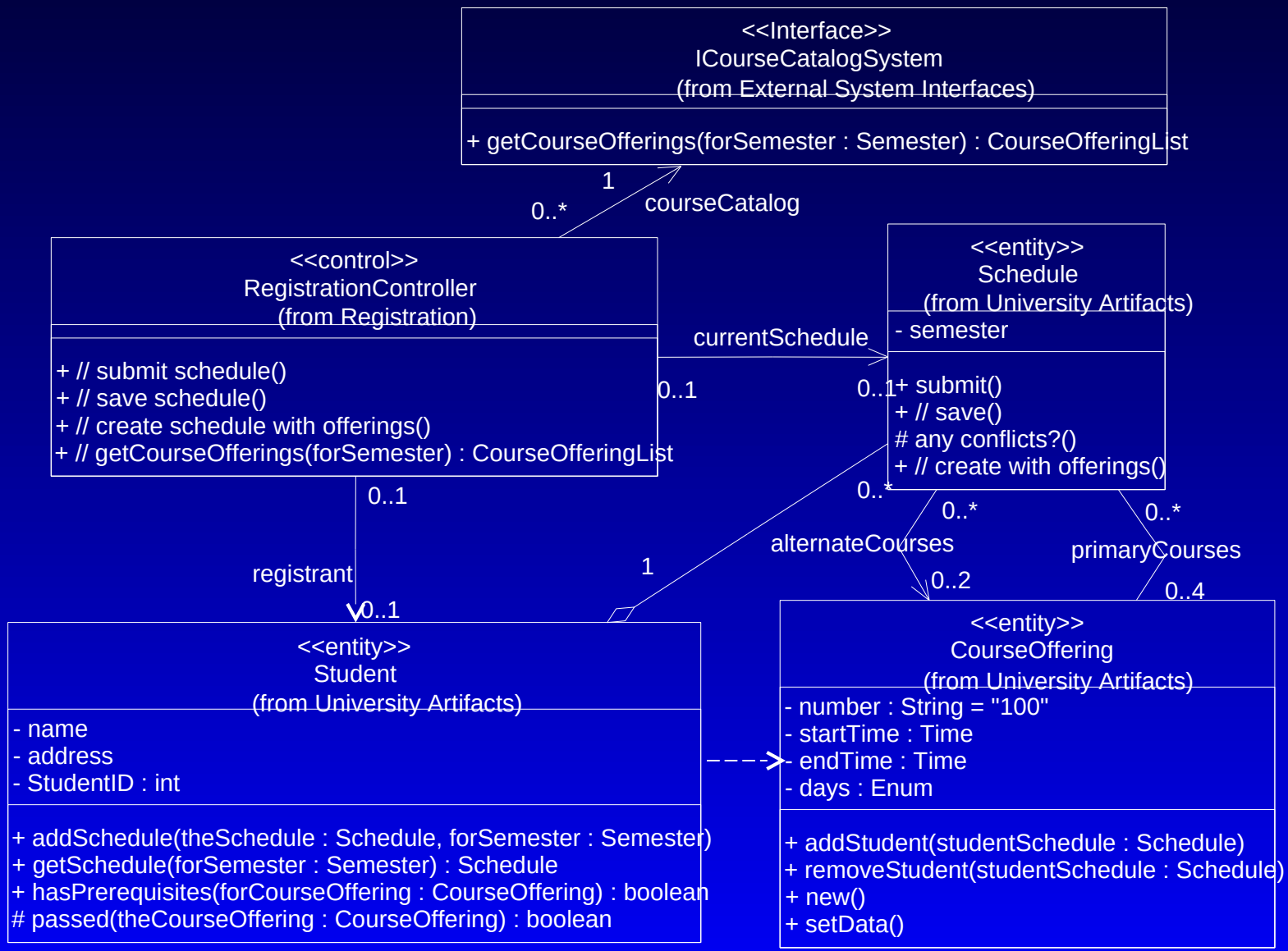
- Strive for real-world relationships
- Strive for the lightest relationships possible (dependency)
- Is relationship “permanent”? Use association (field visibility)
- Is relationship “temporary”? Use dependency
  - Multiple objects share the same instance
    - Pass instance as a parameter (parameter visibility)
    - Make instance a managed global (global visibility)
  - Multiple objects don’t share the same instance (local visibility)
- How long does it take to create/destroy?
  - Expensive? Use field, parameter, or global visibility

Which choice is right? *IT DEPENDS*

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

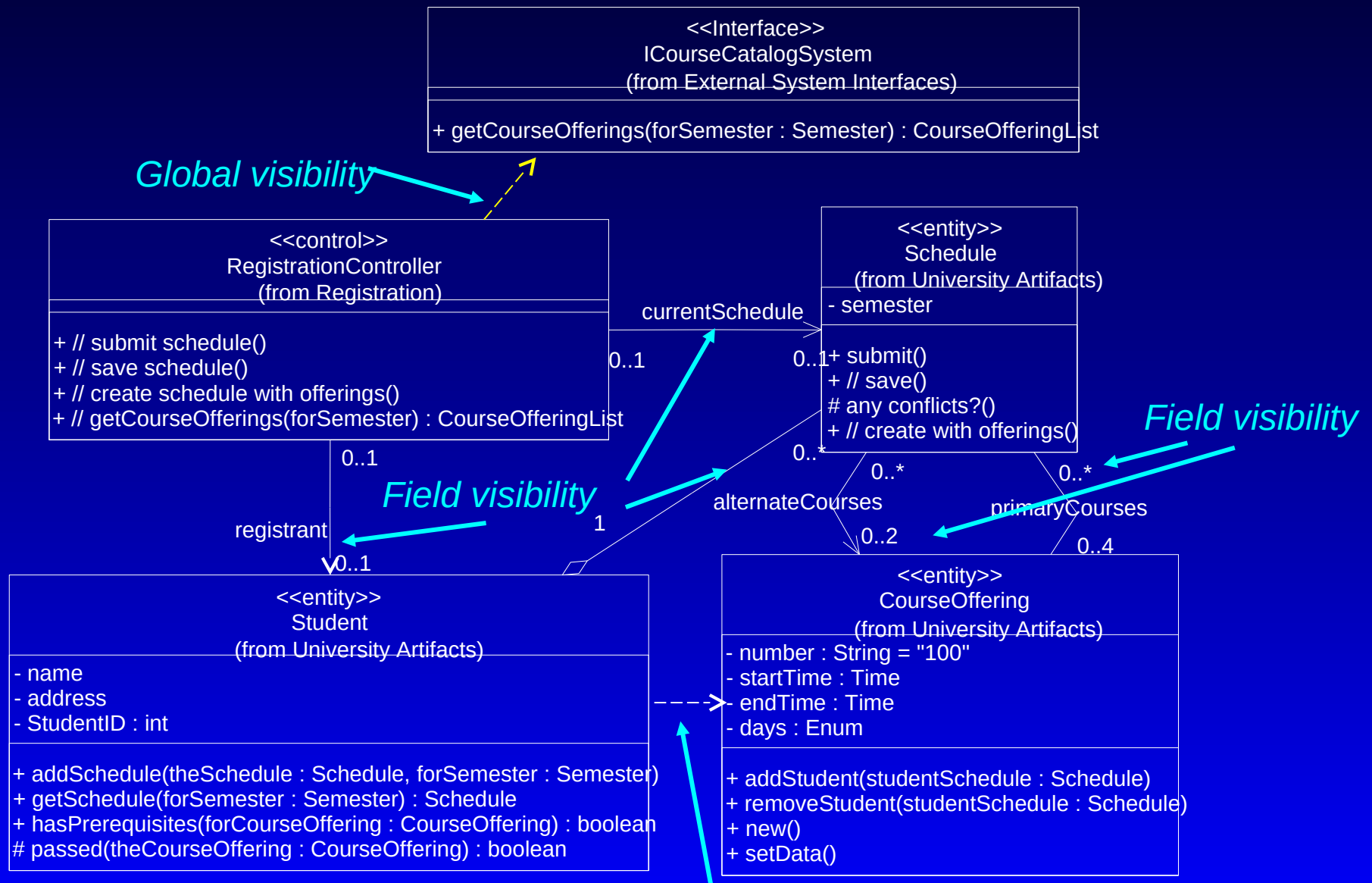
Example: Define Dependencies (before)



Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Example: Define Dependencies (after)



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Define Associations

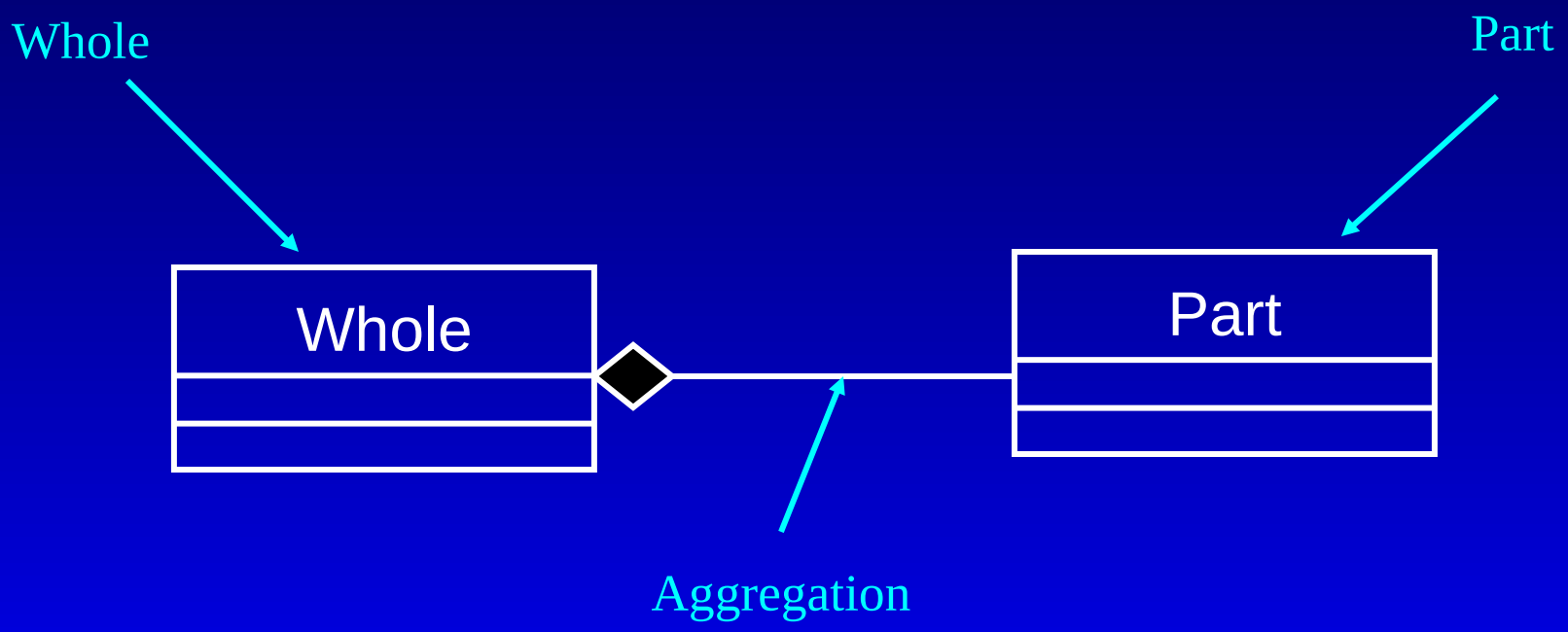
- Purpose
  - Refine remaining associations
- Things to look for :
  - Association vs. Aggregation
  - Aggregation vs. Composition
  - Attribute vs. Association
  - Navigability
  - Association class design
  - Multiplicity design

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Review: What is Composition?

- A form of aggregation with strong ownership and coincident lifetimes
  - The parts cannot survive the whole/aggregate

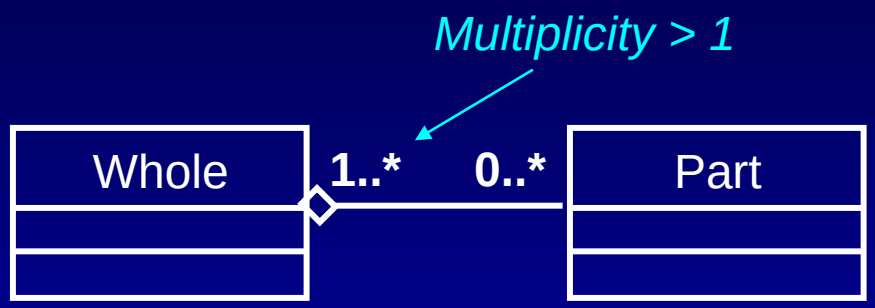


Copyright © 1998-1999 Rational Software, all rights reserved

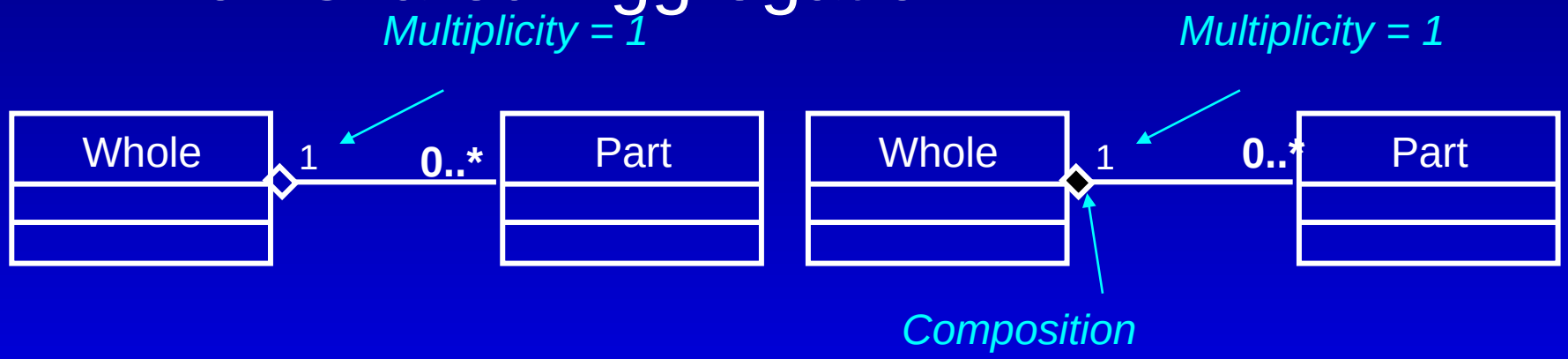
Rational unifying software teams

Aggregation: Shared Vs. Non-shared

- Shared Aggregation



- Non-shared Aggregation



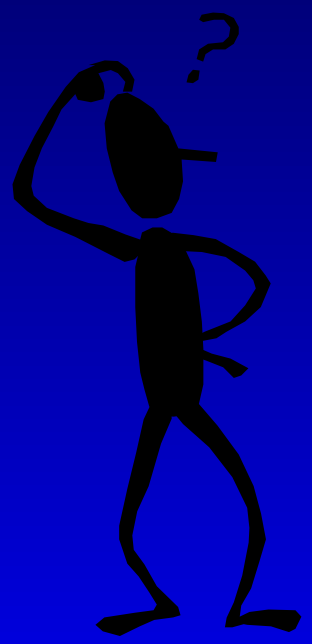
By definition, composition is non-shared aggregation

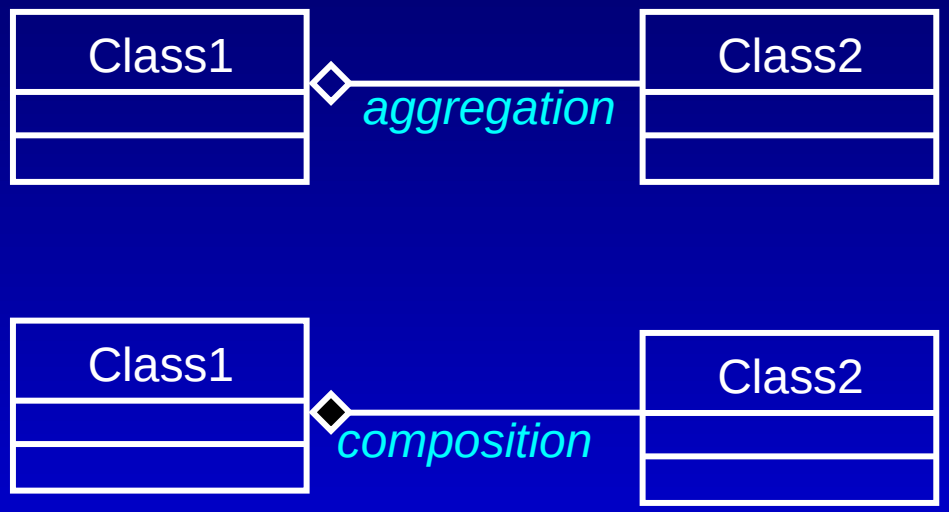
Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Aggregation or Composition?

- Consideration
  - Lifetimes of Class1 and Class2

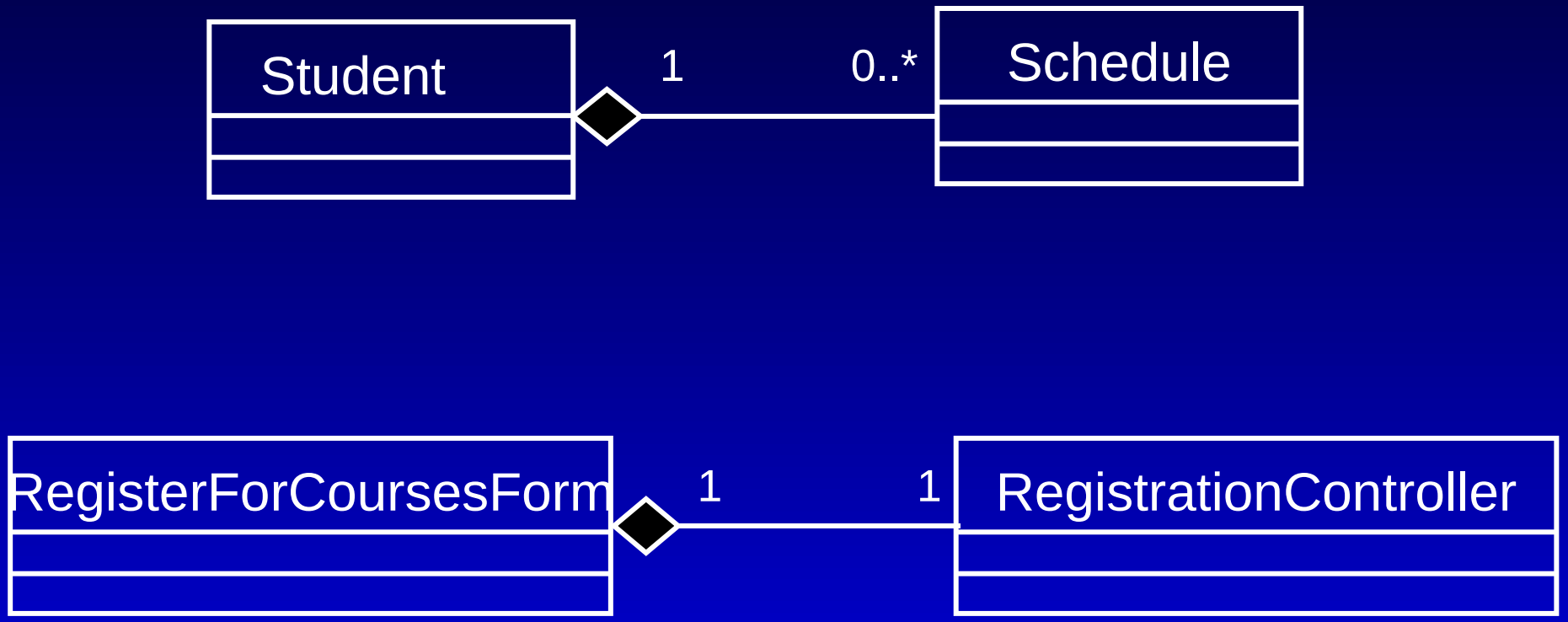




Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Example: Composition



Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

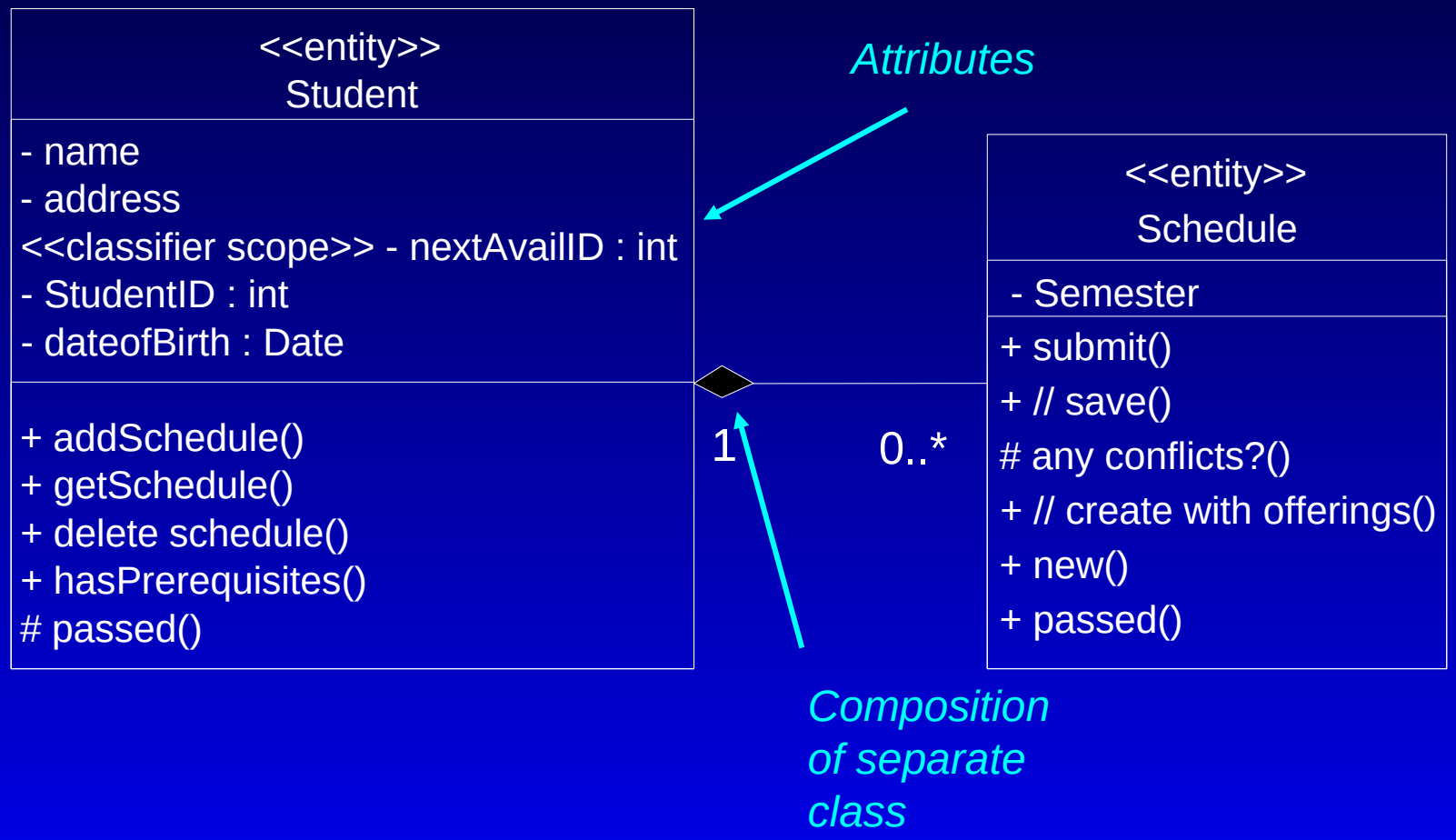
Attributes Vs Composition

- Use composition when
  - Properties need independent identities
  - Multiple classes have the same properties
  - Properties have a complex structure and properties of their own
  - Properties have complex behavior of their own
  - Properties have relationships of their own
- Otherwise use attributes

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Example: Attributes Vs Composition

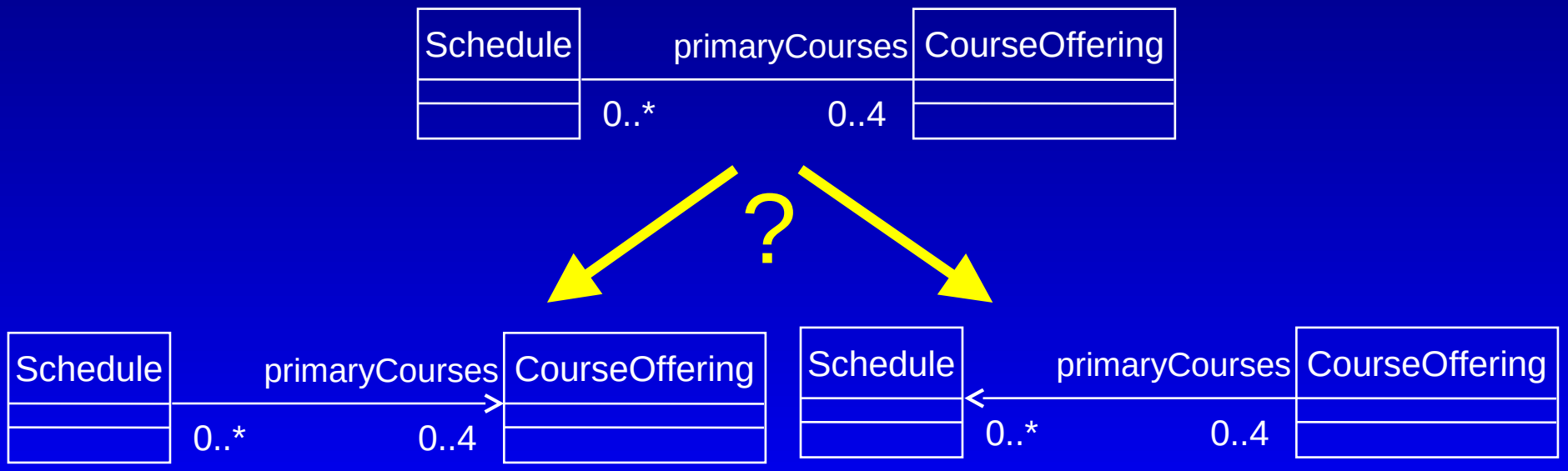


Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Navigability: Which Directions Are Really Needed?

- Explore interaction diagrams
- Even when both directions seem required, one may work
  - Navigability in one direction is infrequent
  - Number of instances of one class is small

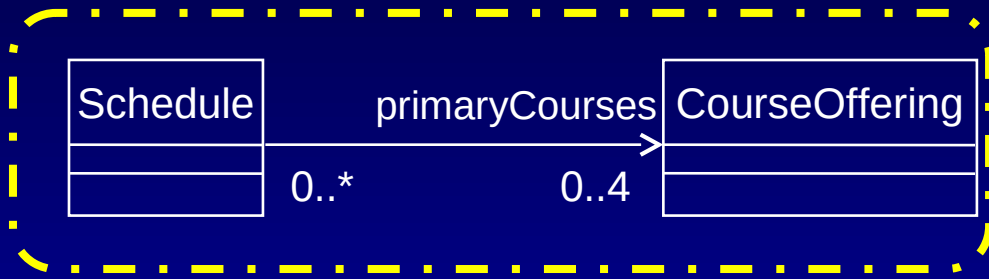


Copyright © 1998-1999 Rational Software, all rights reserved

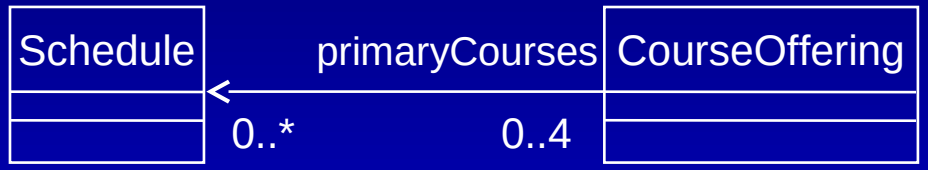
Rational unifying software teams

Example: Navigability Refinement

- Total number of Schedules is small, or
- Never need a list of the Schedules on which the CourseOffering appears



- Total number of CourseOfferings is small, or
- Never need a list of CourseOfferings on a Schedule

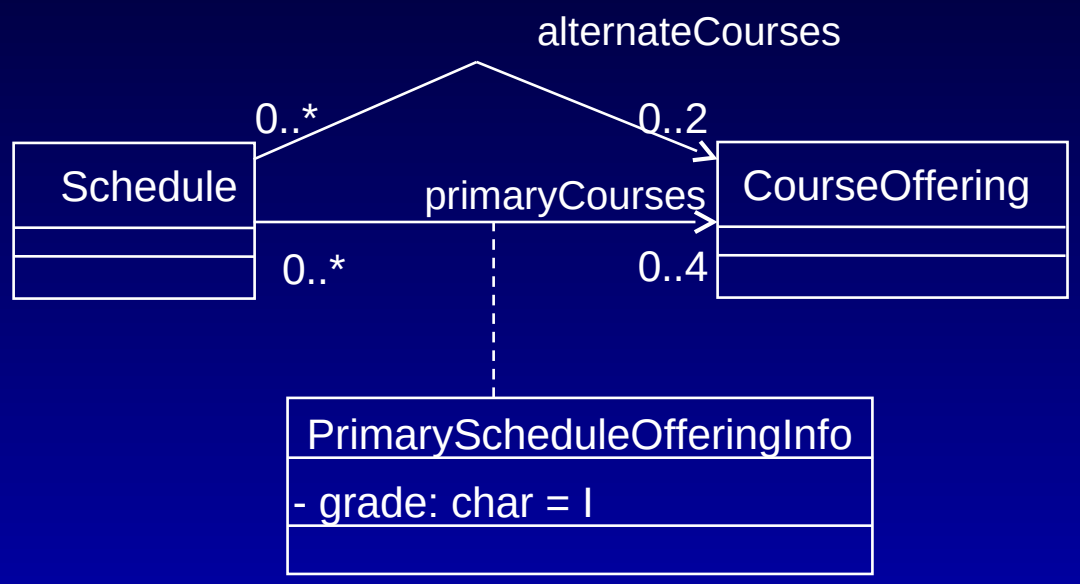


- Total number of CourseOfferings and Schedules are not small
- Must be able to navigate in both directions

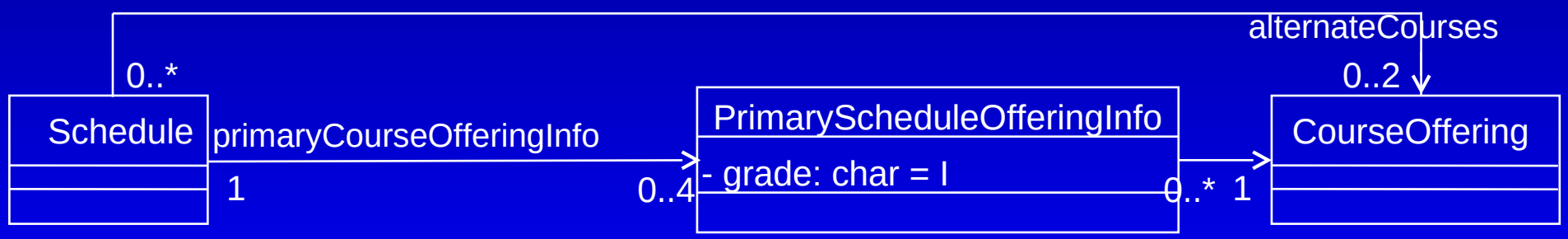
Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Example: Association Class Design



Design Decisions



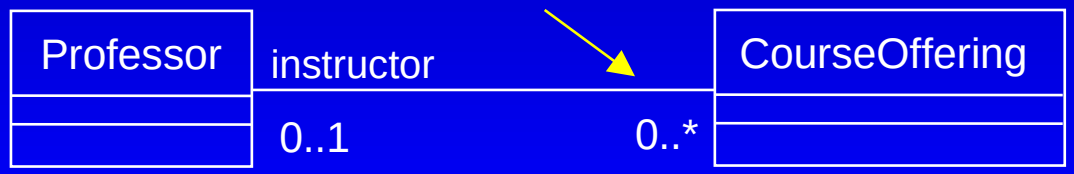
Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Multiplicity Design

- Multiplicity = 1, or Multiplicity = 0..1
  - May be implemented directly as simple value or pointer
  - No further "design" required

- Multiplicity > 0
  - Cannot use a simple value or pointer
  - Further "design" may be required



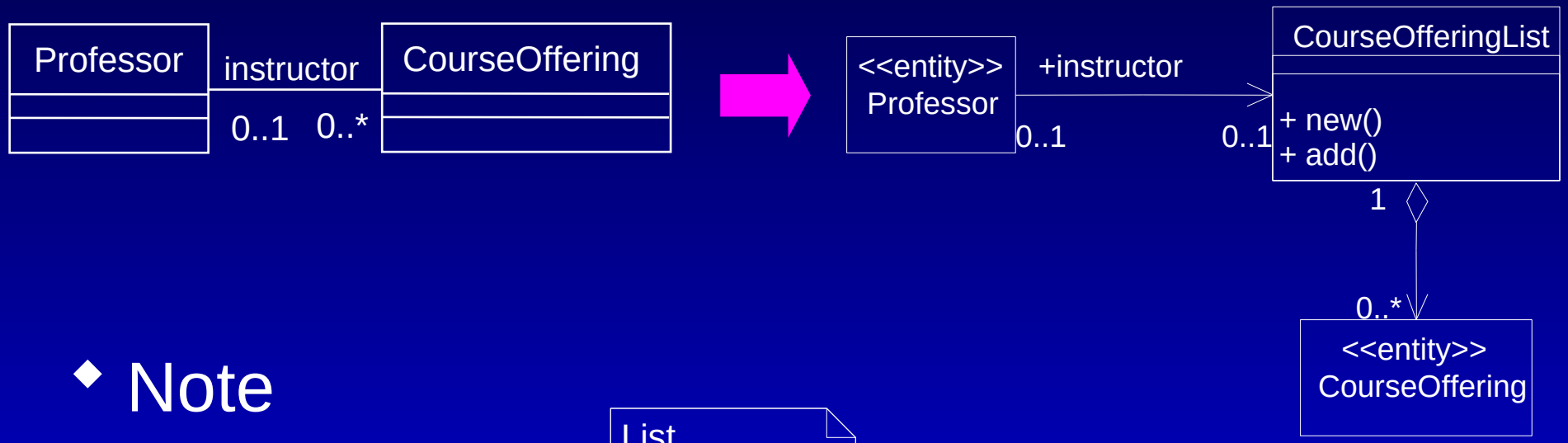
Needs a container

Copyright © 1998-1999 Rational Software, all rights reserved

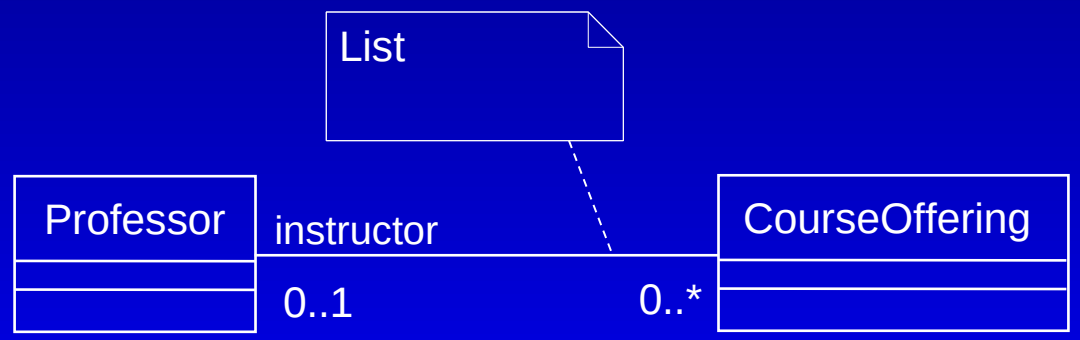
Rational unifying software teams

Multiplicity Design Options

- Explicit modeling of a container class



- Note

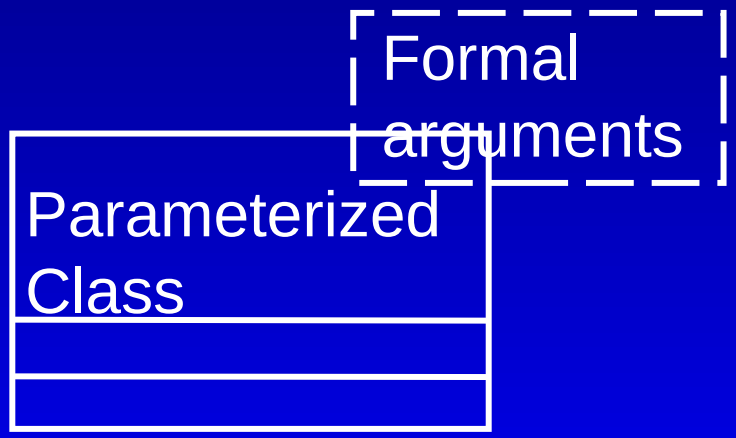


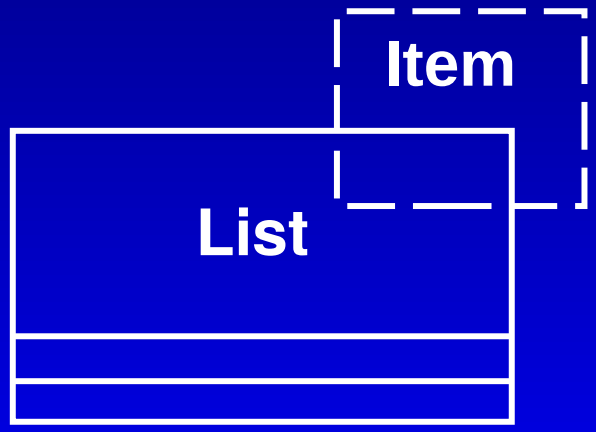
Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

What is a Parameterized Class (template)?

- A class definition which defines other classes
- Often used for container classes
  - Some common container classes:
    - Sets, lists, dictionaries, stacks, queues, …

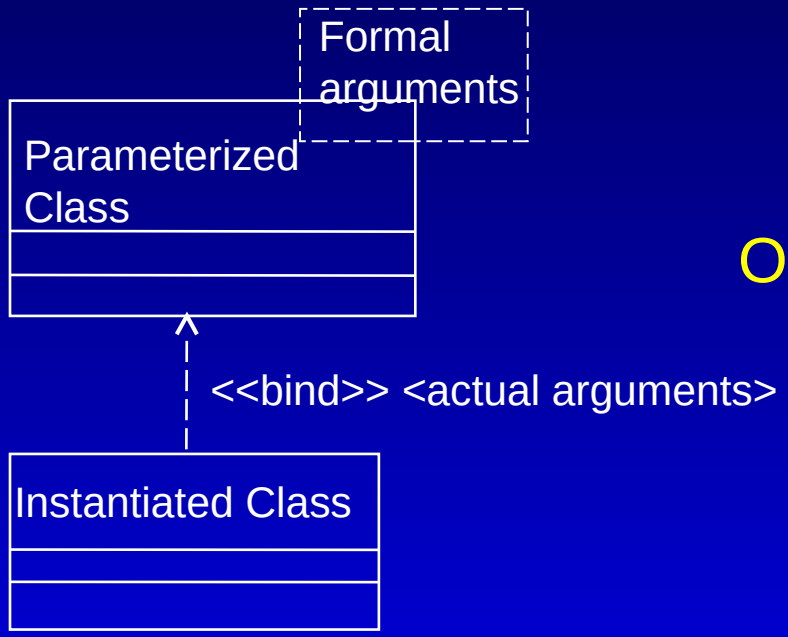




Copyright © 1998-1999 Rational Software, all rights reserved

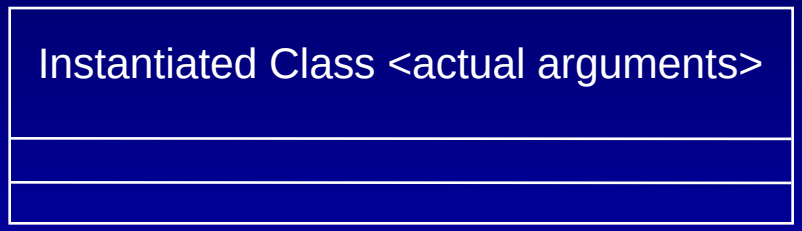
Instantiating a Parameterized Class

Explicit binding



OR

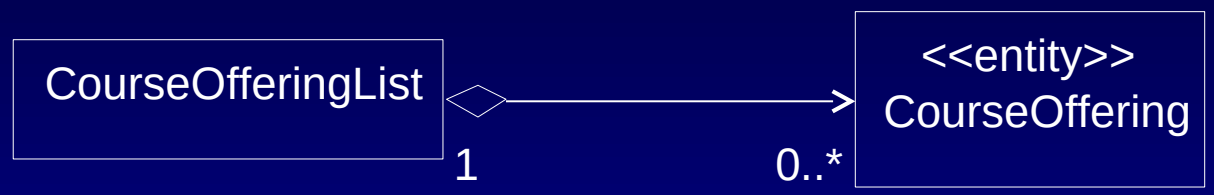
Implicit binding



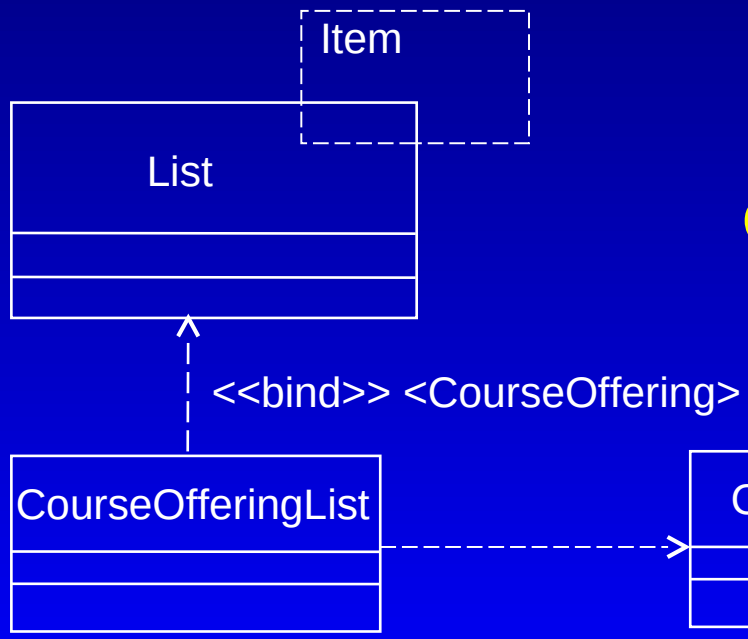
Copyright © 1998-1999 Rational Software, all rights reserved

Example: Instantiating a Parameterized Class

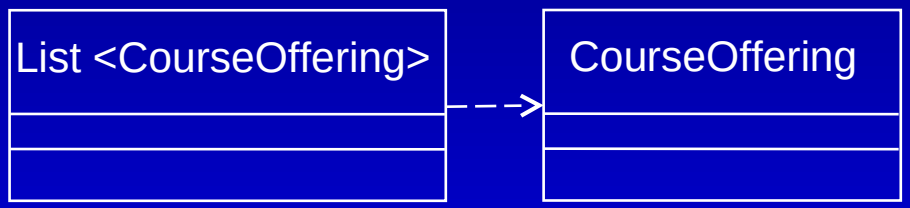
Before



After



OR

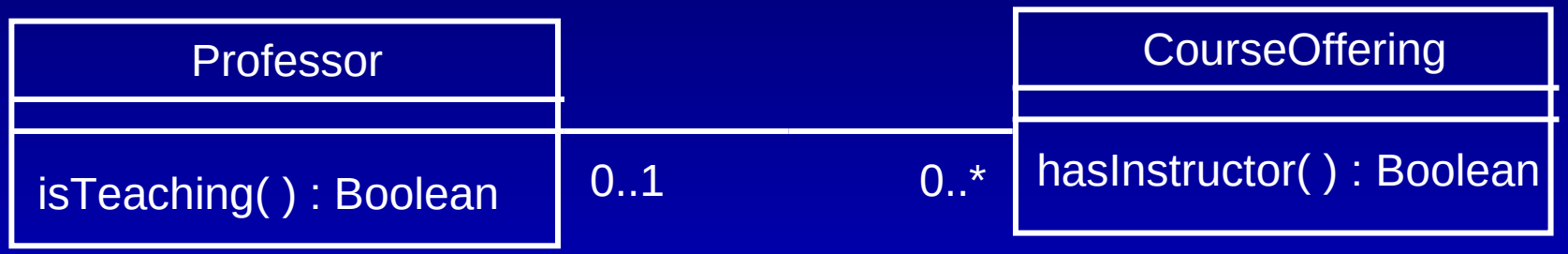


Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Multiplicity Design: Optionality

- If a link is optional, make sure to include an operation to test for the existence of the link



Copyright © 1998-1999 Rational Software, all rights reserved

Exercise: Define Dependencies and Associations

- Given the following:
  - The use-case realization for a use case and/or the detailed design of a subsystem
  - The design of all participating design elements

(continued)

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Exercise: Define Dependencies and Associations (cont.)

- Identify the following:
  - The required navigability for each relationship
  - Any additional classes to support the relationship design
  - Any associations refined into dependencies
  - Any associations refined into aggregations or compositions
  - Any refinements to multiplicity

(continued)

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Exercise: Define Dependencies and Associations (cont.)

- Produce the following diagrams
  - An updated VOPC, including the relationship refinements

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Define Generalizations

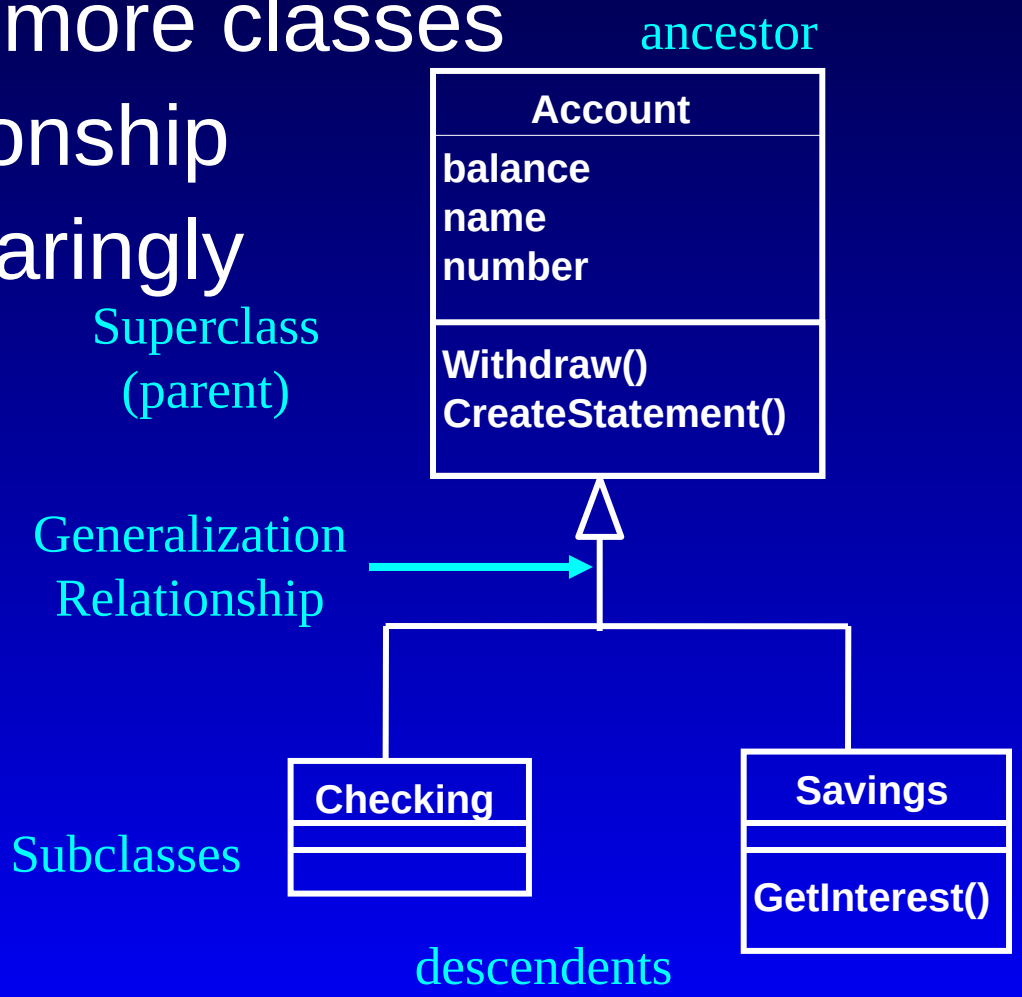
- Purpose
  - Identify areas of reuse
  - Refine existing inheritance hierarchies so that they can be implemented efficiently
- Things to look for:
  - Abstract classes vs. concrete classes
  - Multiple inheritance problems
  - Generalization vs. Aggregation
  - Generalization to support implementation reuse
  - Generalization to support polymorphism
  - Generalization to support metamorphosis
  - Generalization simulation

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Review: Generalization

- One class shares the structure and/or behavior of one or more classes
- "Is-a-kind of" relationship
- In analysis, use sparingly

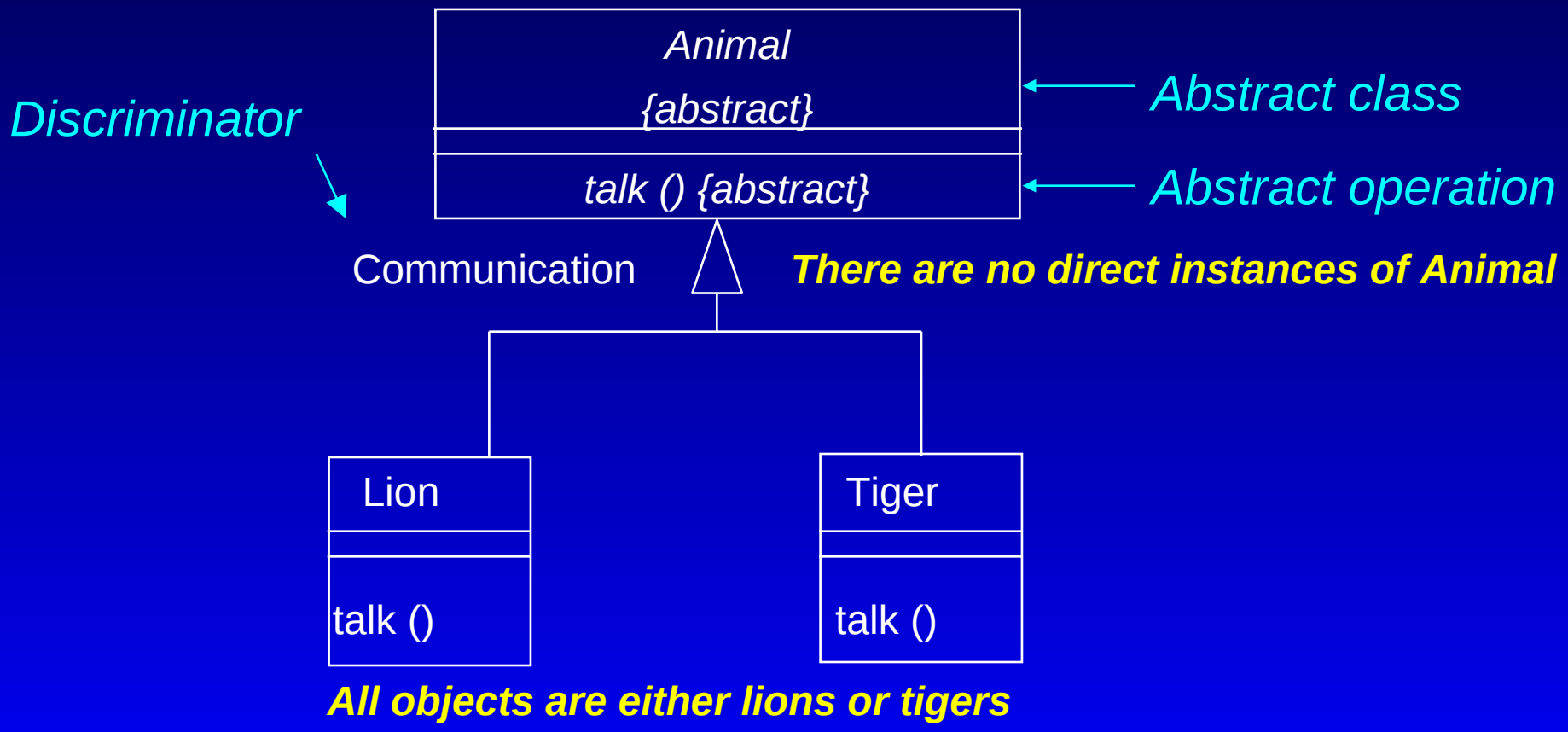


Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Abstract and Concrete Classes

- Abstract classes cannot have any objects
- Concrete classes can have objects

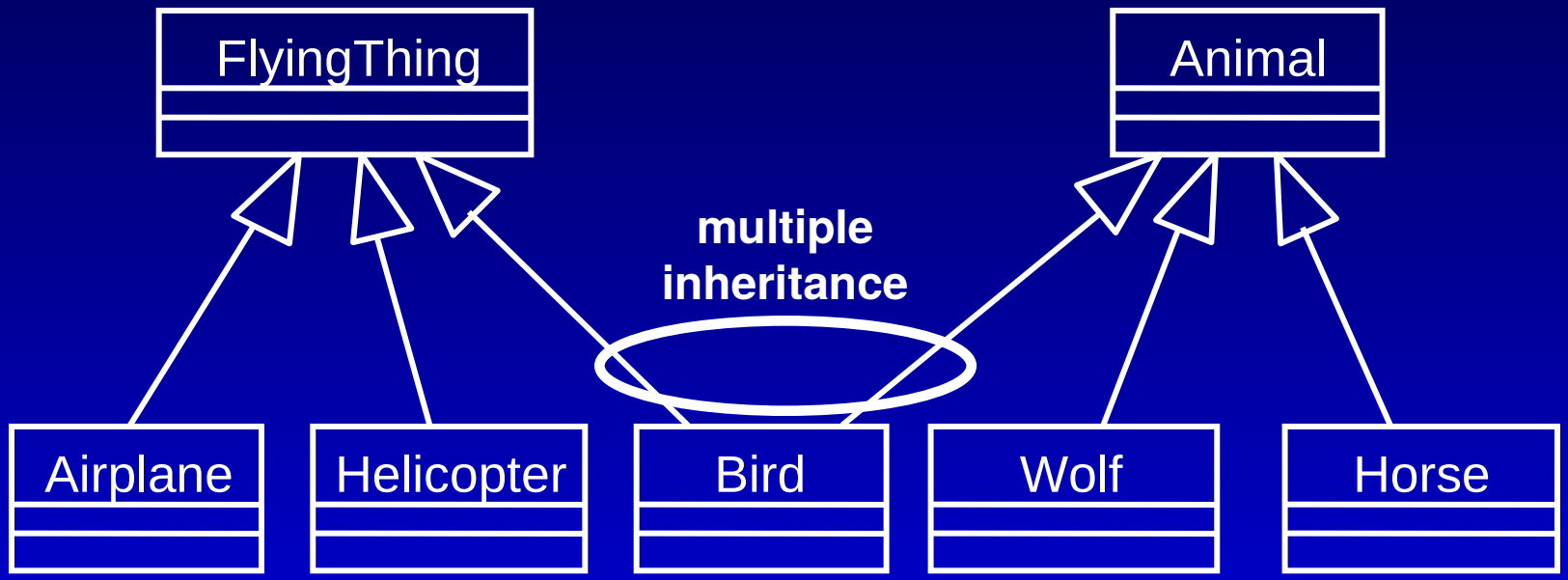


Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Review: Multiple Inheritance

- A class can inherit from several other classes



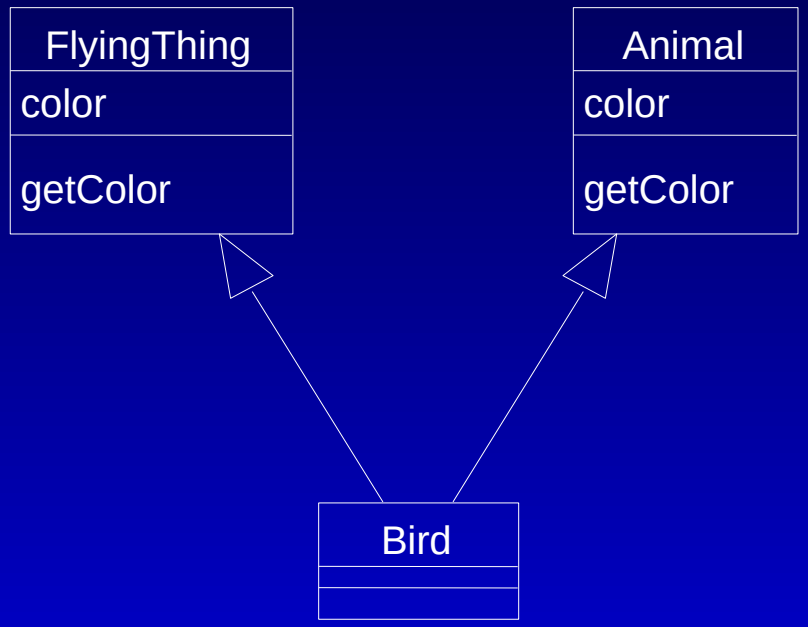
Use multiple inheritance only when needed, and always with caution !

Copyright © 1998-1999 Rational Software, all rights reserved

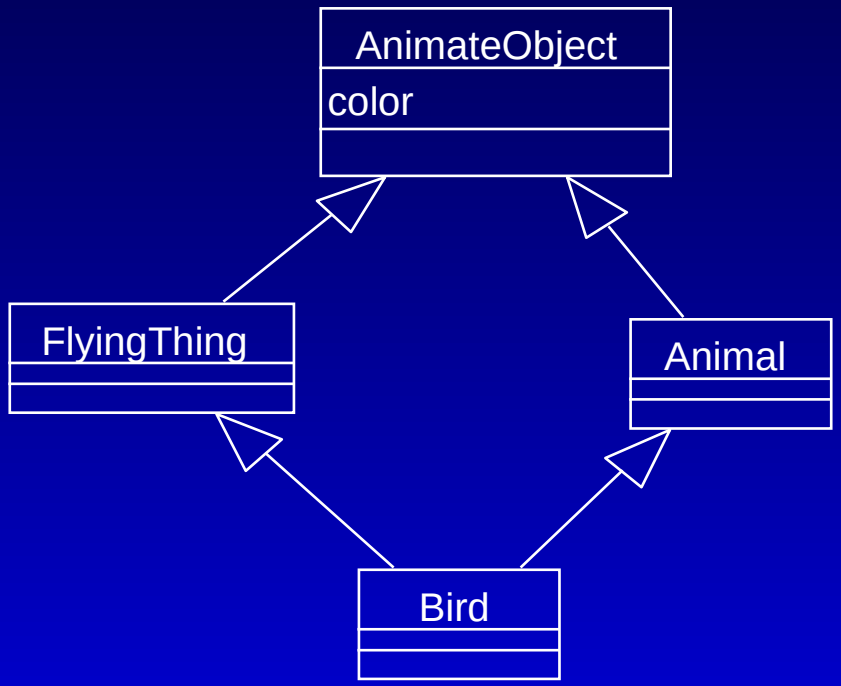
Rational
unifying software teams

Multiple Inheritance: Problems

Name clashes on attributes or operations



Repeated inheritance



Resolution of these problems is implementation-dependent

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

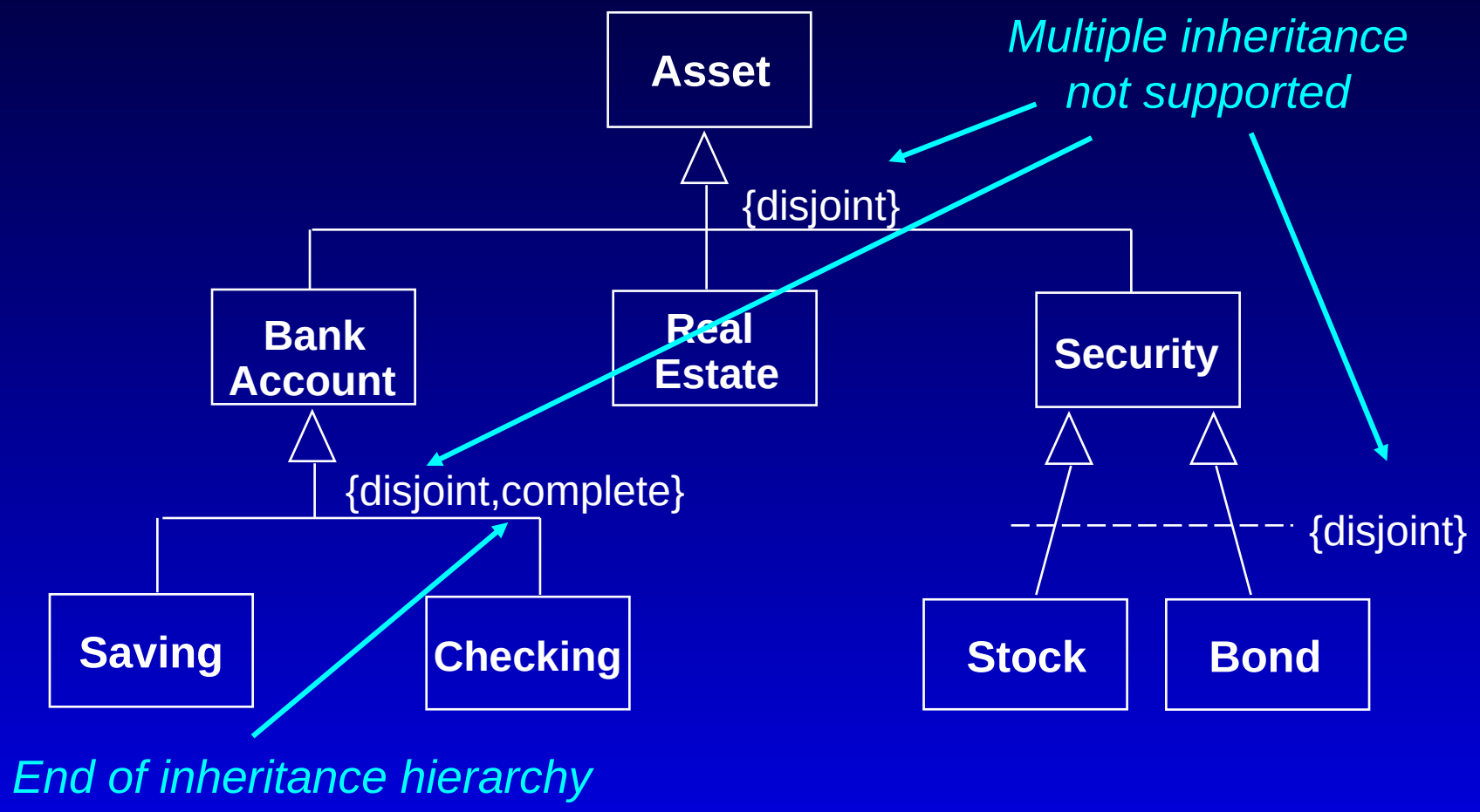
Generalization Constraints

- Complete
  - End of the inheritance tree
- Incomplete
  - Inheritance tree may be extended
- Disjoint
  - Subclasses mutually exclusive
  - Doesn’t support multiple inheritance
- Overlapping
  - Subclasses are not mutually exclusive
  - Supports multiple inheritance

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Example: Generalization Constraints

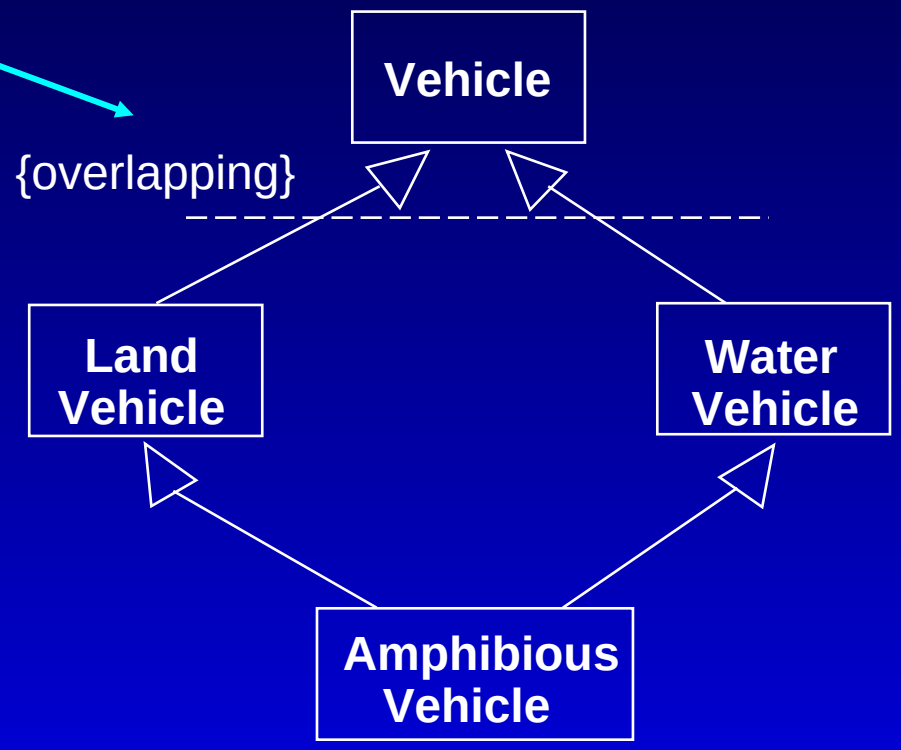


Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Example: Generalization Constraints (cont.)

Multiple inheritance supported

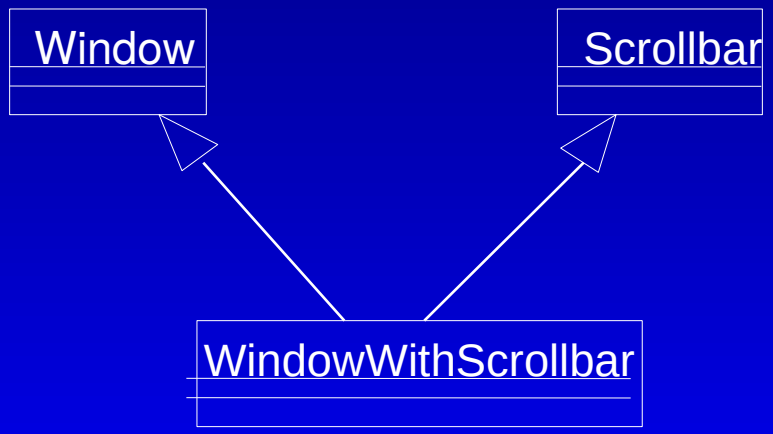


Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Generalization vs. Aggregation

- Generalization and aggregation are often confused
  - Generalization represents an “is-a” or “kind-of” relationship
  - Aggregation represents a “part-of” relationship

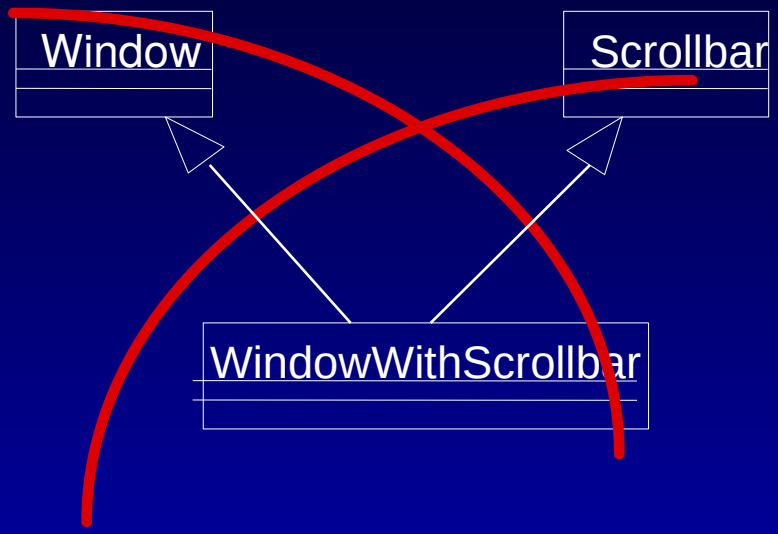


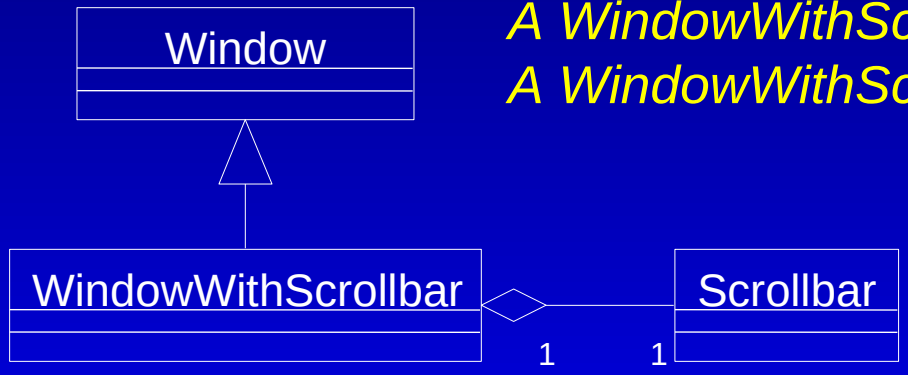
Is this correct?

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Generalization vs. Aggregation





A WindowWithScrollbar "is a" Window
A WindowWithScrollbar "contains a" Scrollbar

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Generalization Uses

- Share Common Properties and Behavior
- Share Implementation
- Implement Polymorphism
- Implement Metamorphosis

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Generalization Uses

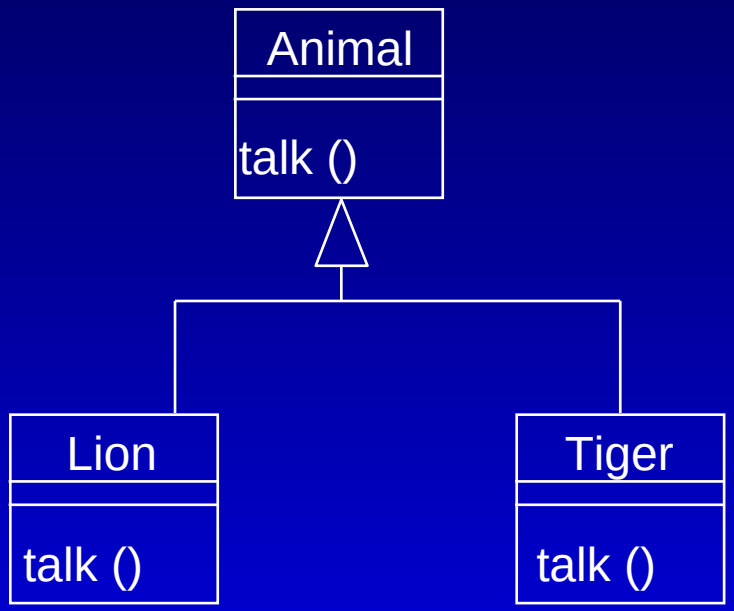
- Share Common Properties and Behavior
- Share Implementation
- Implement Polymorphism
- Implement Metamorphosis

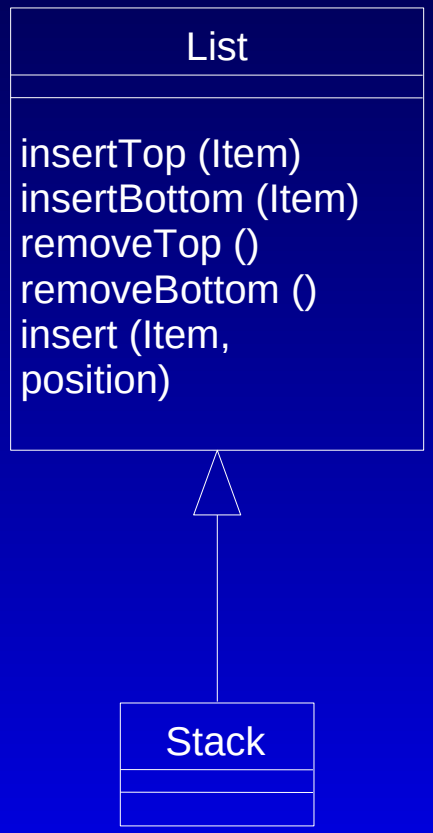
Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Generalization: Share Common Properties and Behavior

- Follows the Is-A style of programming
- Class substitutability



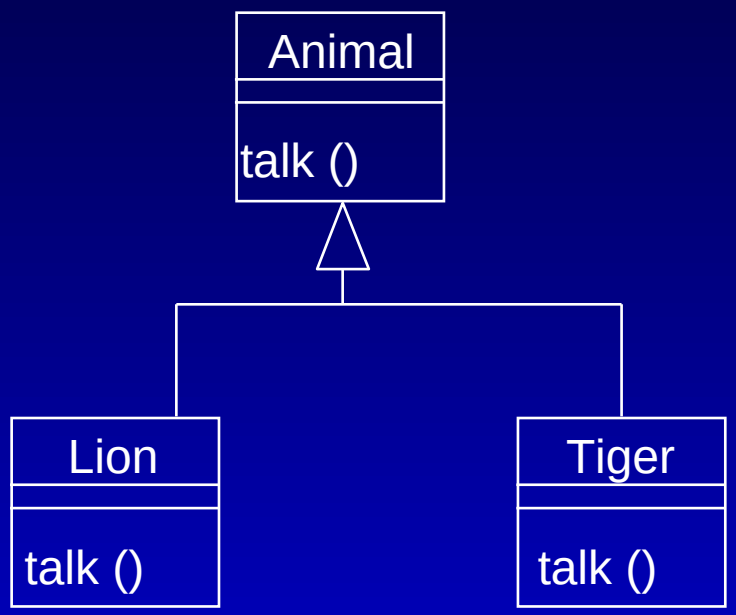


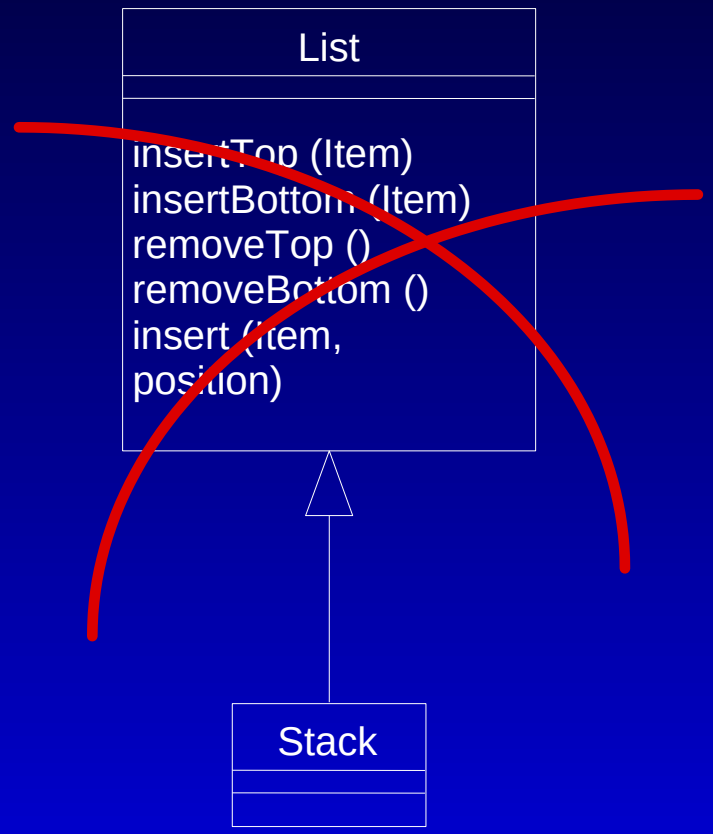
Do these classes follow the IS-A style of programming?

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Generalization: Share Common Properties and Behavior (contd)





Copyright © 1998-1999 Rational Software, all rights reserved

Generalization Uses

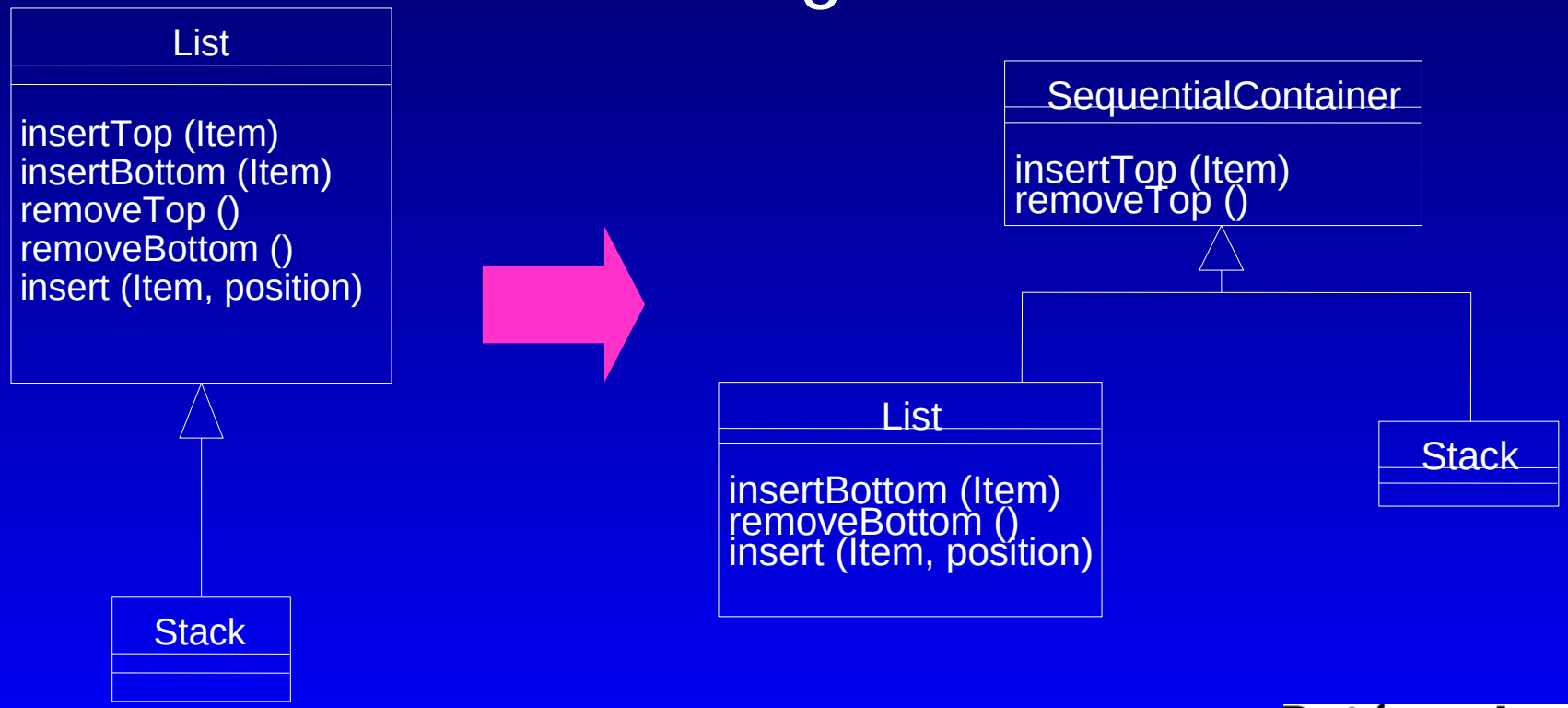
- Share Common Properties and Behavior
- Share Implementation
- Implement Polymorphism
- Implement Metamorphosis

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Generalization: Share Implementation: Factoring

- Supports the reuse of the implementation of another class
- Cannot be used if the class you want to "reuse" cannot be changed

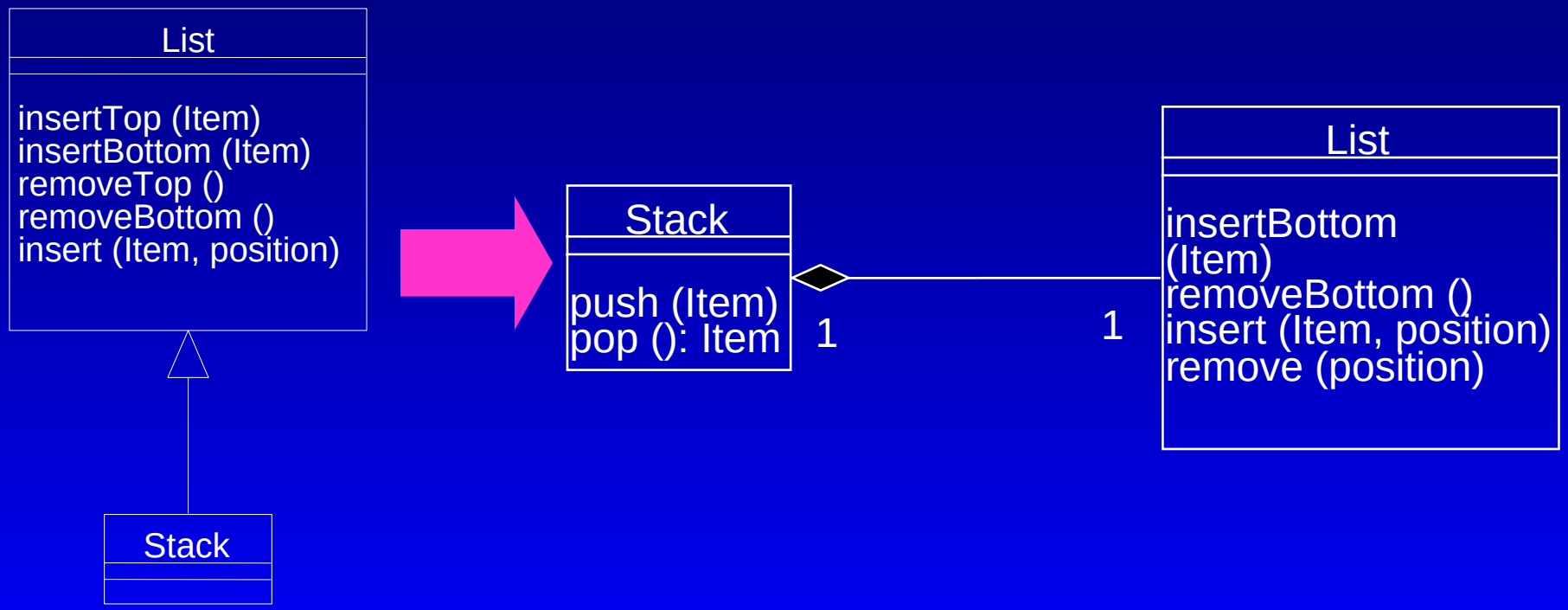


Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Generalization Alternative: Share Implementation: Delegation

- Supports the reuse of the implementation of another class
- Can be used if the class you want to "reuse" cannot be changed



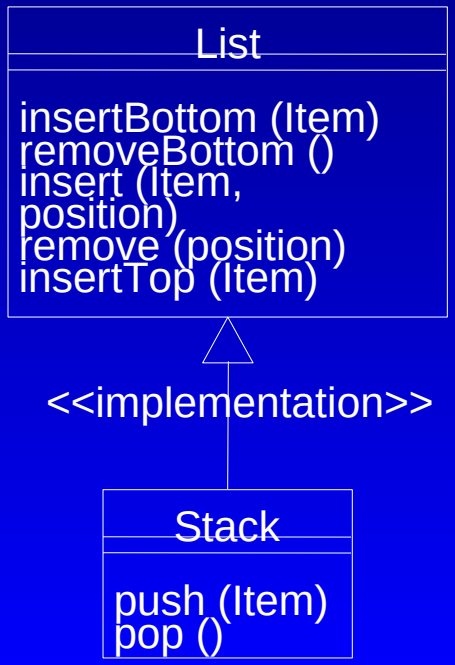
Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Implementation Inheritance

- Ancestor public operations, attributes and relationships are NOT visible to clients of descendent class instances
- Descendent class must define all access to ancestor operations, attributes, and relationships

push() and pop() can access methods of List but instances of Stack cannot



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Generalization Uses

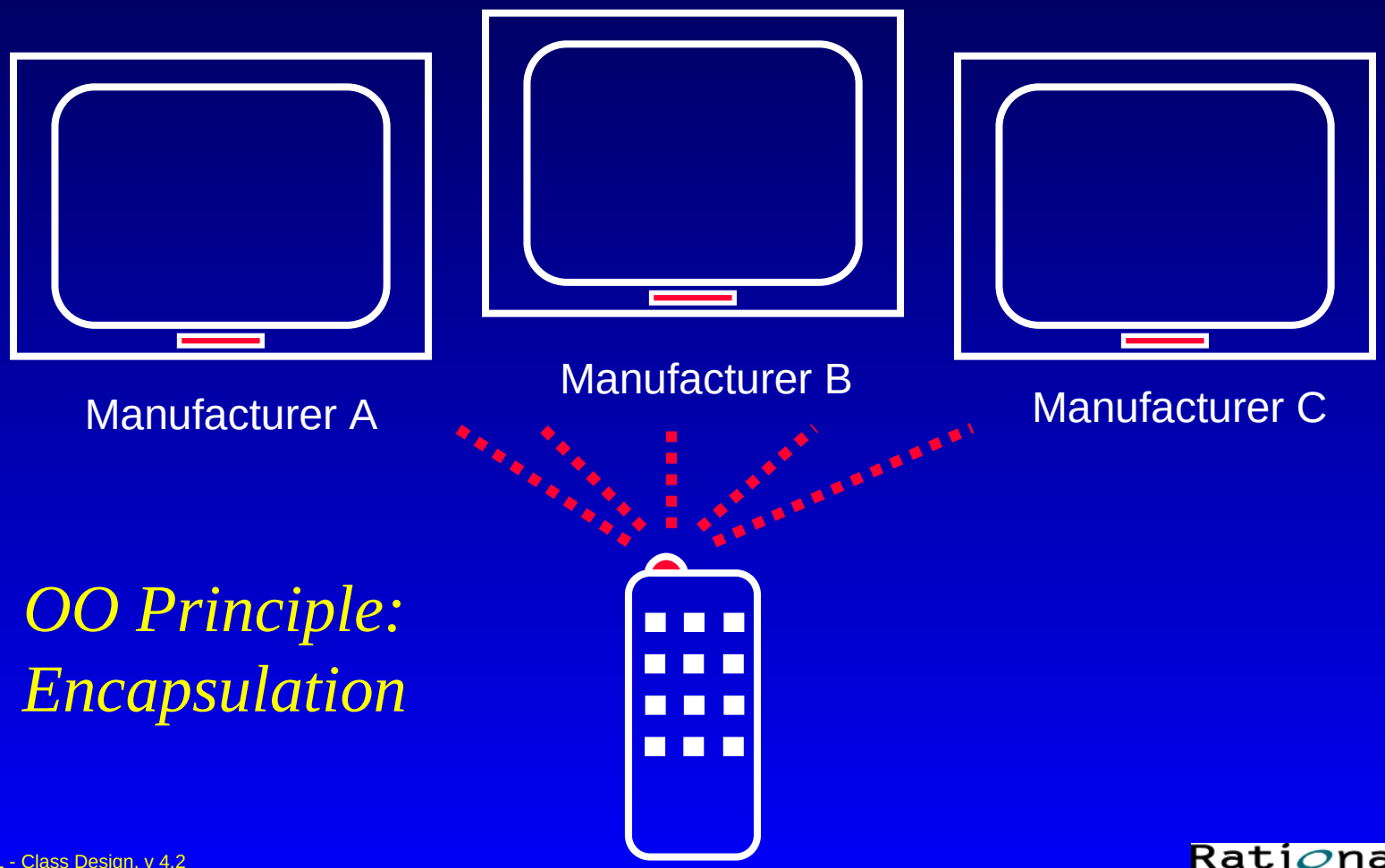
- Share Common Properties and Behavior
- Share Implementation
- Implement Polymorphism
- Implement Metamorphosis

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Review: What is Polymorphism?

- The ability to hide many different implementations behind a single interface

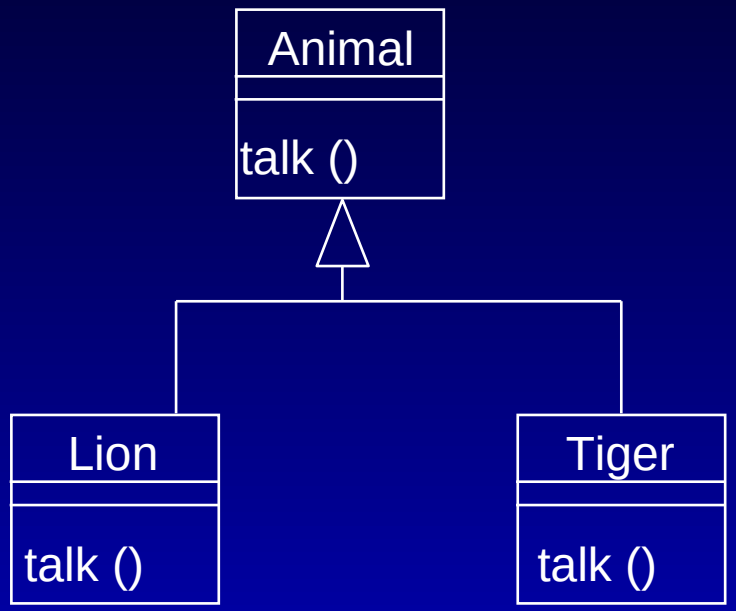


OO Principle: Encapsulation

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Generalization: Implement Polymorphism



Without Polymorphism

if animal = “Lion” then
do the Lion talk
else if animal = “Tiger” then
do the Tiger talk
end

With Polymorphism

do the Animal talk

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Polymorphism: Use of Interfaces vs. Generalization

- Interfaces support implementation-independent representation of polymorphism
  - Realization relationships can cross generalization hierarchies
- Interfaces are pure specifications, no behavior
  - Abstract base class may define attributes and associations
- Interfaces are totally independent of inheritance
  - Generalization is used to re-use implementations
  - Interfaces are used to re-use behavioral specifications
- Generalization provides a way to implement polymorphism

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Polymorphism Via Generalization Design Decisions

- Provide interface only to descendant classes?
  - Design ancestor as an abstract class
  - All methods provided by descendent classes
- Provide interface and default behavior to descendent classes?
  - Design ancestor as a concrete class with a default method
  - Allow polymorphic operations
- Provide interface and mandatory behavior to descendent classes?
  - Design ancestor as a concrete class
  - Do not allow polymorphic operations

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Generalization Uses

- Share Common Properties and Behavior
- Share Implementation
- Implement Polymorphism
- Implement Metamorphosis

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

What is Metamorphosis?

- Metamorphosis
  - 1. a change in form, structure, or function; specifically the physical change undergone by some animals, as of the tadpole to the frog
  - 2. any marked change, as in character, appearance, or condition
  - Webster's New World Dictionary, Simon & Schuster, Inc., 1979

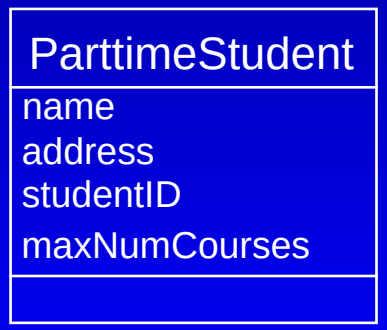
Metamorphosis exists in the real world
How should it be modeled?

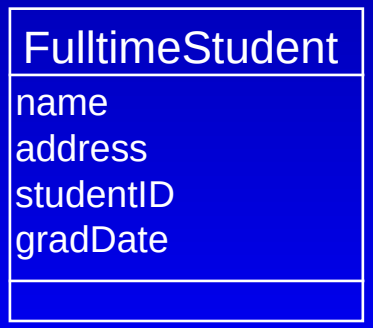
Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Example: Metamorphosis

- In the university, there are full time students and part time students
  - Full time students have an expected graduation date but part time students do not
  - Part time students may take a maximum of 3 courses where there is no maximum for full time students



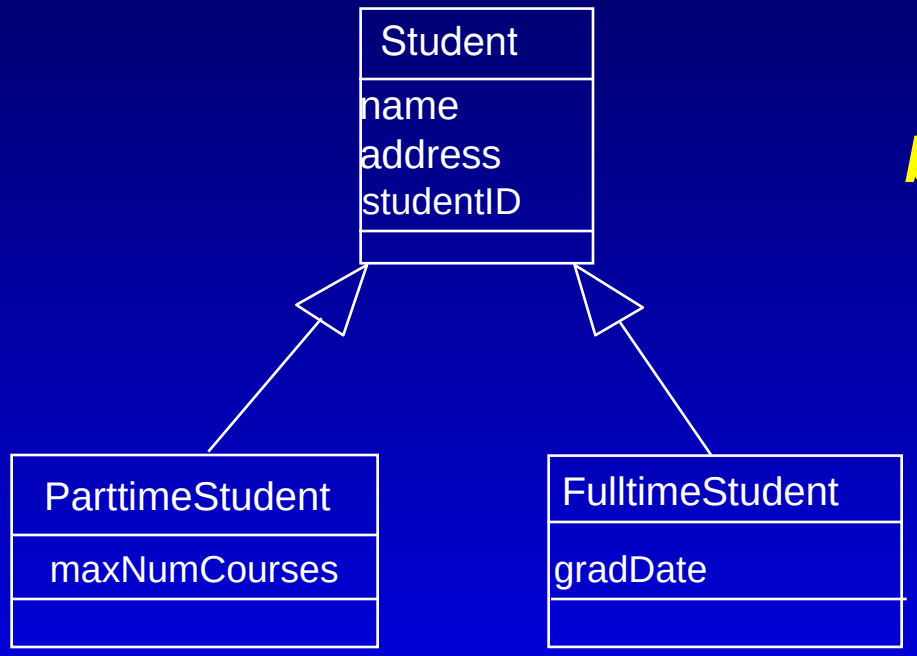


Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Modeling Metamorphosis: One Approach

- An generalization relationship may be created



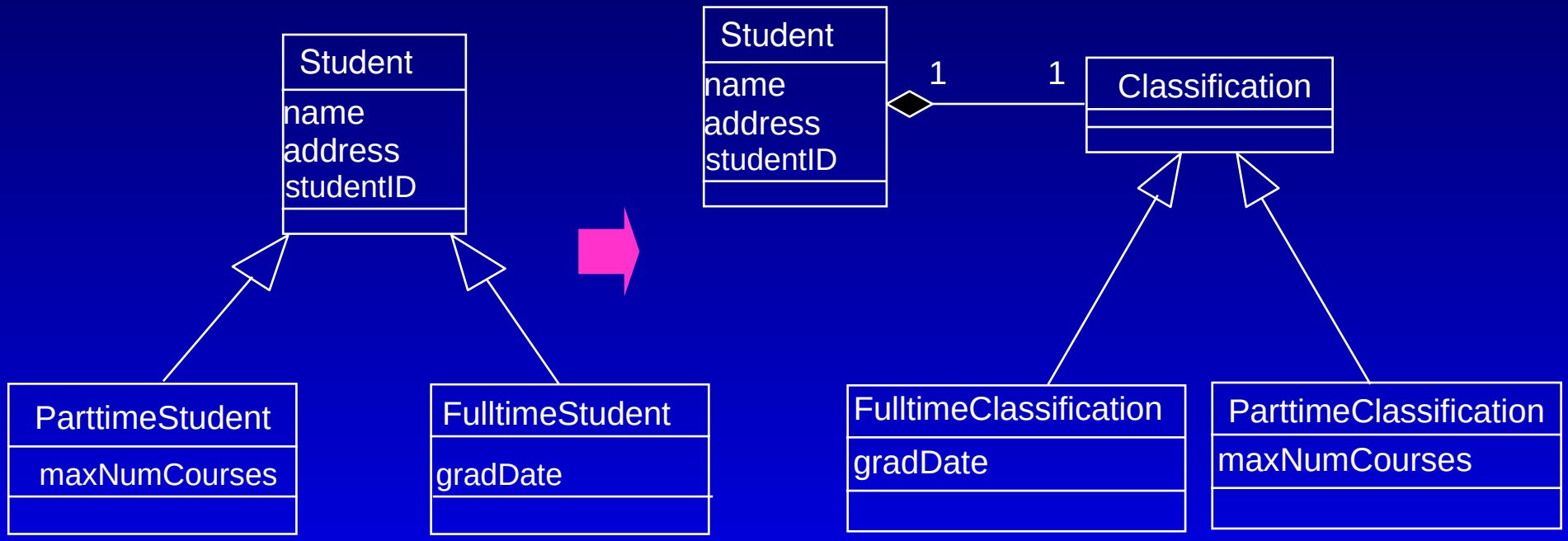
What happens if a part-time student becomes a full-time student?

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Modeling Metamorphosis: Another Approach

- Inheritance may be used to model common structure, behavior and/or relationships to the “changing” parts

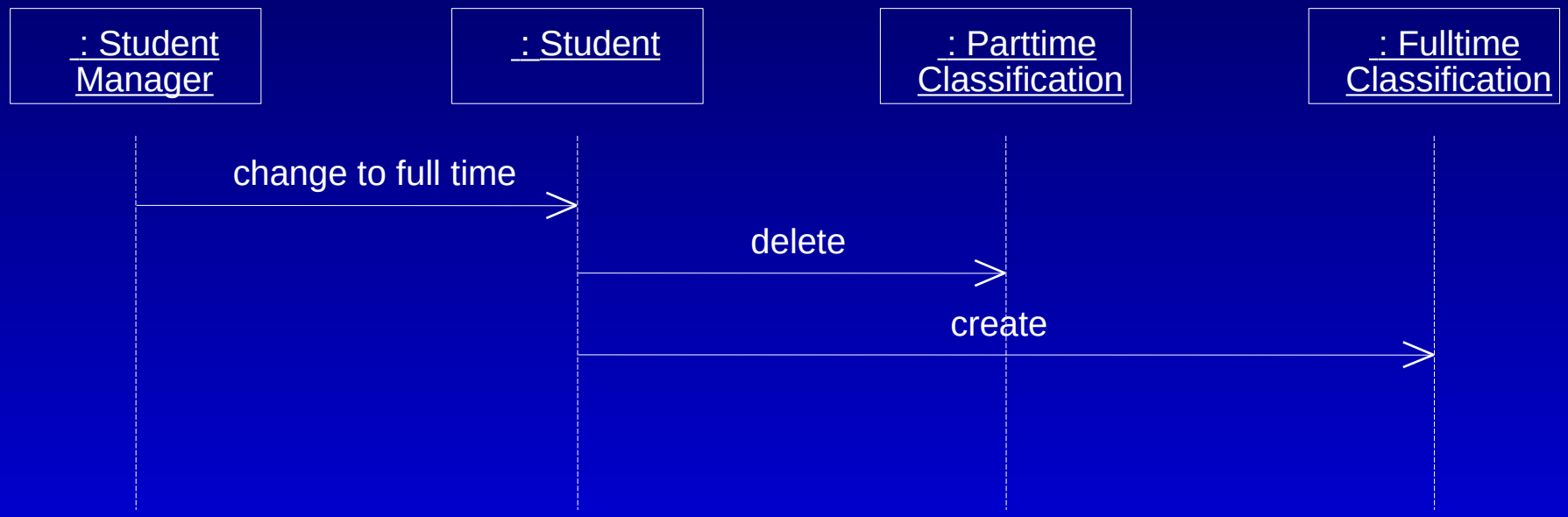


Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Modeling Metamorphosis: Another Approach (contd)

- Metamorphosis is accomplished by the object “talking” to the changing parts

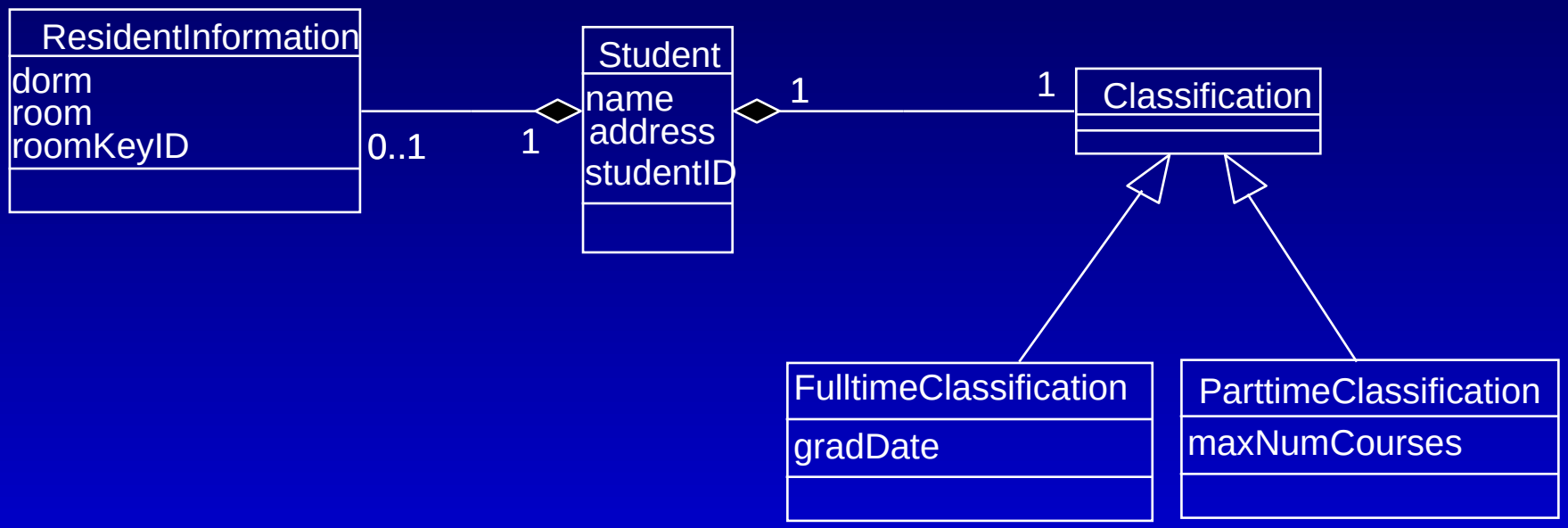


Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Metamorphosis and Flexibility

- This technique also adds to the flexibility of the model



Copyright © 1998-1999 Rational Software, all rights reserved

Exercise: Define Generalizations

- Given the following:
  - All design classes
- Identify the following:
  - Any refinements to existing generalizations
  - Any new applications of generalization
    - Make sure any metamorphosis is considered
- Produce the following diagrams:
  - Class diagram containing any new and/or refined generalization relationships between classes
  - (optional) Refined use-case realizations

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Resolve Use-Case Collisions

- Multiple use cases may simultaneously access design objects
- Options
  - Use synchronous messaging => first-come first-serve order processing
  - Identify operations (or code) to protect
  - Apply access control mechanisms
    - Message queuing
    - Semaphores (or 'tokens')
    - Other locking mechanism
- Resolution is highly dependent on implementation environment

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

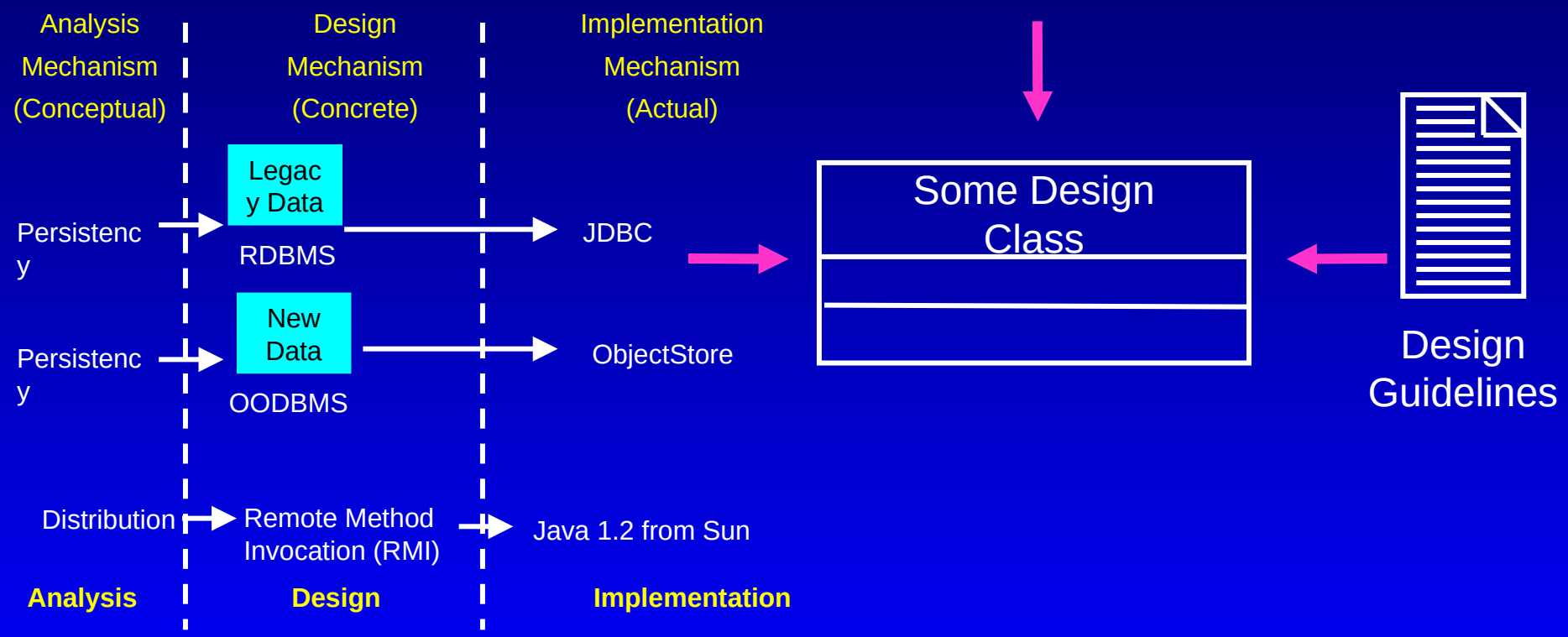
- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Handle Non-Functional Requirements in General

Analysis Class	Analysis Mechanism(s)
Student	Persistency, Security
Schedule	Persistency, Security
CourseOffering	Persistency, Legacy Interface
Course	Persistency, Legacy Interface
RegistrationController	Distribution



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Class Design Steps

- Create Initial Design Classes
- Identify Persistent Classes
- Define Operations
- Define Class Visibility
- Define Methods
- Define States
- Define Attributes
- Define Dependencies
- Define Associations
- Define Generalizations
- Resolve Use-Case Collisions
- Handle Non-Functional Requirements in General
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Checkpoints: Classes

- Does the name of each class clearly reflect the role it plays?
- Does the class represent a single well-defined abstraction?
- Are all attributes and responsibilities functionally coupled?
- Are there any class attributes, operations or relationships that should be generalized, that is, moved to an ancestor?
- Are all specific requirements on the class addressed?
- Are the demands on the class consistent with any statecharts which model the behavior of the class and its instances?
- Is the complete life cycle of an instance of the class described?
- Does the class offer the required behavior?

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Checkpoints: Operations

- Are the operations understandable?
- Is the state description of the class and its objects' behavior correct?
- Does the class offer the behavior required of it?
- Have you defined the parameters correctly
- Have you assigned operations for the messages of each object completely?
- Are the implementation specifications (if any) for an operation correct?
- Do the operation signatures conform to the standards of the target programming language?
- Are all the operations needed by the use-case realizations?

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Checkpoints: Attributes

- Does each attribute represent a single conceptual thing?
- Are the names of the attributes descriptive?
- Are all the attributes needed by the use-case realizations?

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Checkpoints: Relationships

- Are the role names descriptive?
- Are the multiplicities of the relationships correct?

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Review: Class Design

- What is the purpose of Class Design?
- In what ways are classes refined?
- Are statecharts created for every class?
- What are the major components of a statechart? Provide a brief description of each.
- What kind of relationship refinements occur?
- What is the difference between an association and a dependency?
- What is done with operations and attributes?

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams